

Pipeline
March 4th to March 25th

**Embedded
Systems**

Pipeline Group 2.0

- ❖ Review and General Updates
- ❖ Stage Division
 - Overview
 - Memory Considerations
 - Stage Implementation
 - Pipes
 - Top Module Implementation
- ❖ Hazards
 - Load-Use Hazards
 - Data Hazards (RAW)
 - Control Hazards
- ❖ CPI
- ❖ Conclusion
 - Next Steps
 - Possible Issues

01

Review & General Updates

Rewind to these weeks' next steps

Objectives for the upcoming weeks

Divide the work to implement each stage as a separate subsystem

- ❖ Implement each stage of the pipeline as a separate module.
- ❖ Implement each pipeline register as a separate module.
- ❖ Integrate all the module subsystems to form the 5-stage pipeline.

Implement the hazard solution mechanisms (data + control)

- ❖ Implement the hazard unit.
 - Data forwarding, load stalls.
 - Control branch prediction, data forwarding.
- ❖ Wire the hazard logic into the pipeline.

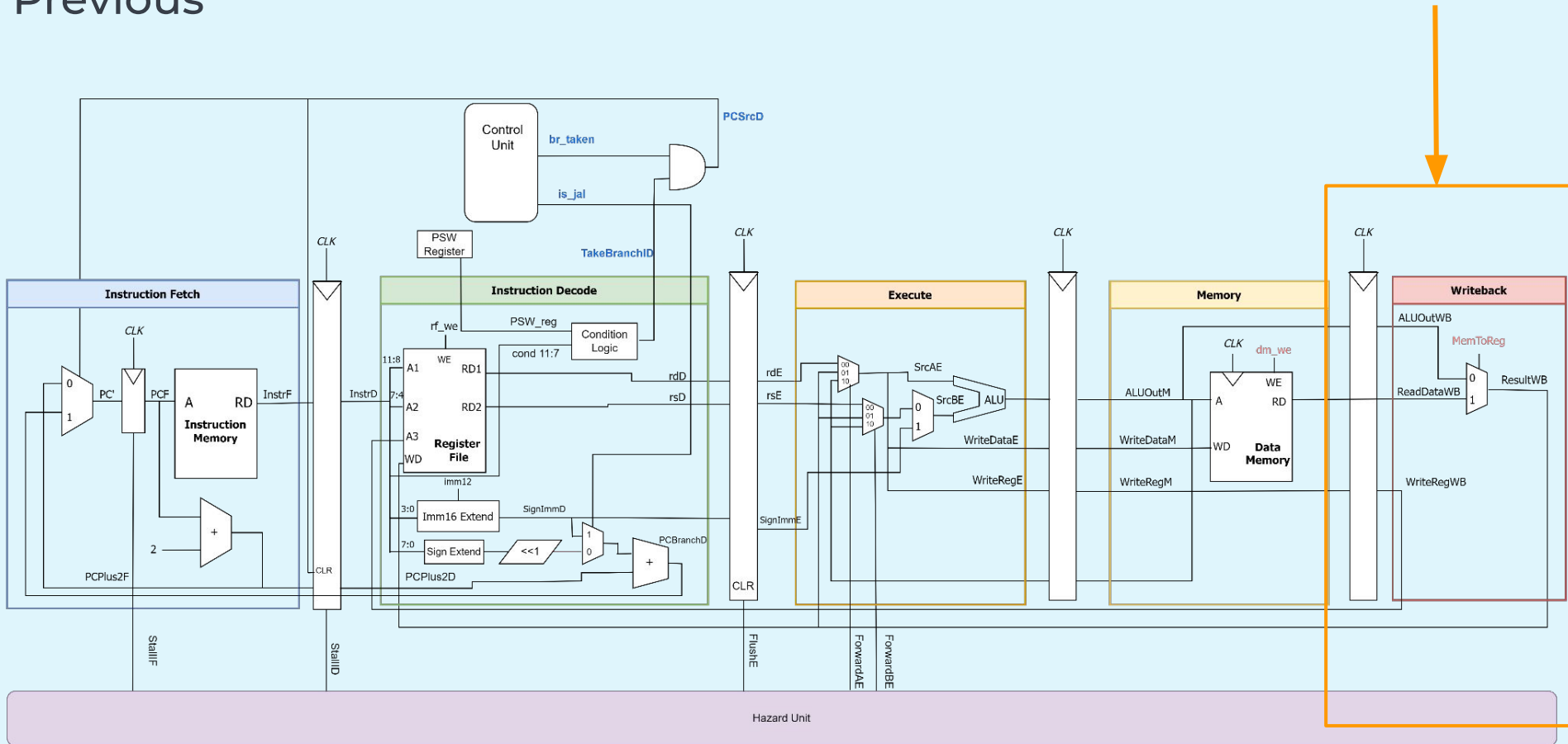
And, as suggested by the professor, evaluate if we needed to have the WB stage or not.

Pipeline Status

- ❖ Implemented all the pipeline stages
 - Connected them, dealing with the hazards (RAW, load-use and taken-branch) with stalls and flushes
- ❖ Implemented the hazard unit, with forwarding control
- ❖ Implemented *early branch* prediction + *predict-not-taken* logic
- ❖ Validated, in the behavioral simulation, all the hazards and the processor's correct behavior
- ❖ Calculated the CPI of the pipelined processor vs the previous non-pipelined processor

Design – Pipeline Architecture

Previous



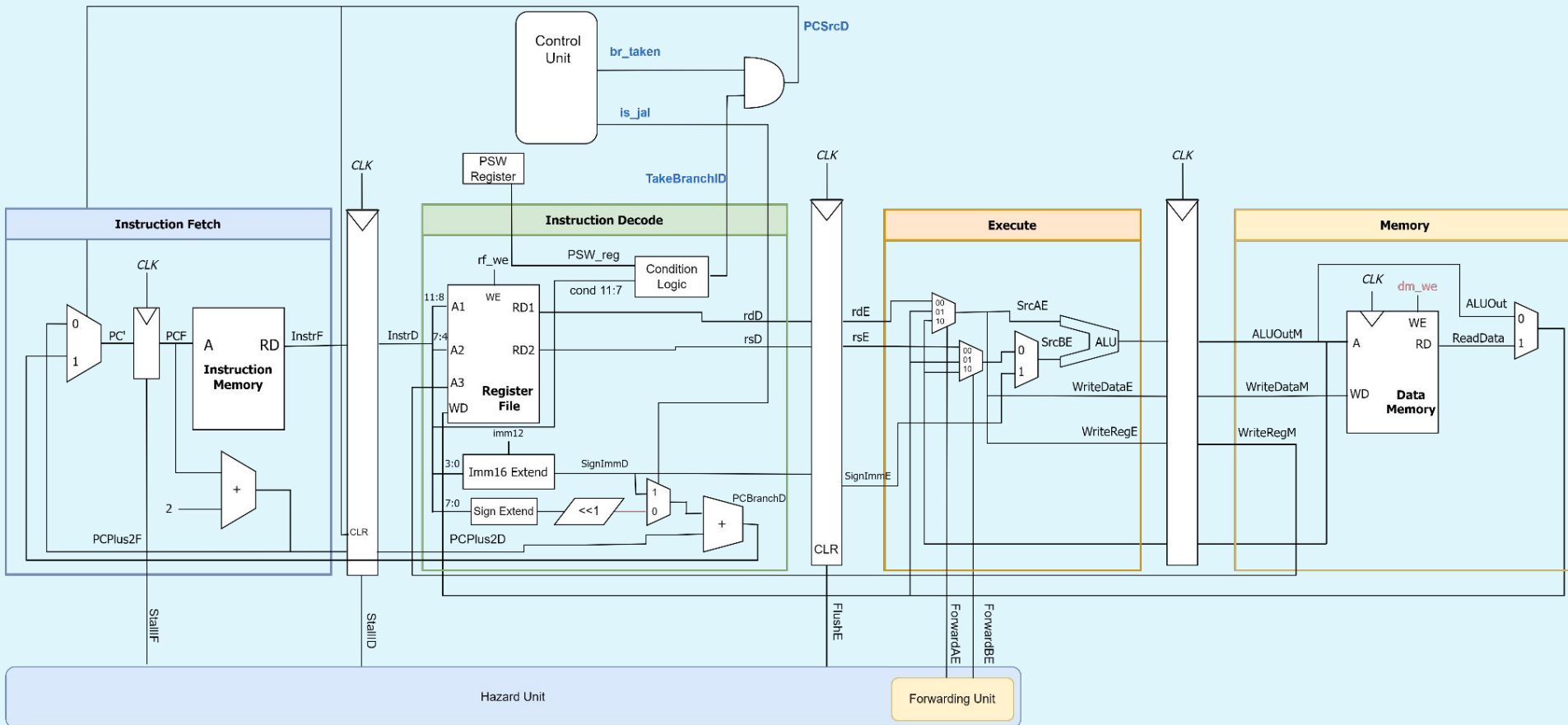
Writeback stage or no writeback stage?

Given that the WB stage was purely stateless (no registers – only combinational logic), and did not have any strong logical implications in any other part of the pipeline structure we opted to **remove the WB stage** and to move its logic to MEM stage.

Result: Faster processor. Lower latency. No need to have forwarding from WB stage, some RAW hazards will resolve sooner.

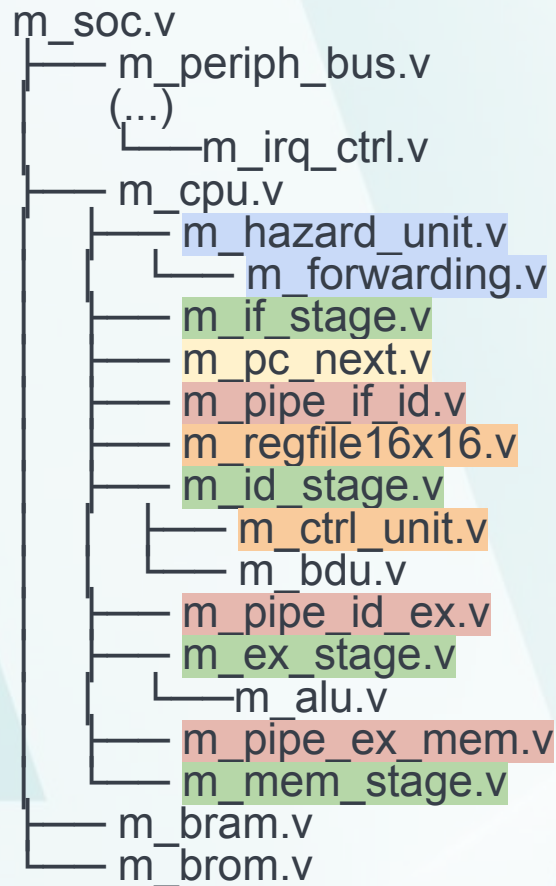
Design – Pipeline Architecture

Current



Disclaimer: This design does not include all control signals from the Control Unit!

Project Hierarchy & Organization



Github branches

pipeline/5-stage-with-stalls

pipeline/5-stage-with-forwarding

pipeline/4-stage

Current branch

Located under **.../assembly/**

▼ assembly

 branch_00.asm

 branch_01.asm

 forwarding_00.asm

 forwarding_01.asm

 input.asm

 jump_branch_test0.a...

 load-use_stall_00.asm

Example

```
; =====  
; 0x0114 - main loop (16 iterations, 4 branches each)  
; =====  
loop:  
; branch 1: BEQ not taken (t0=0 != t1=1 → Z=0)  
CMP  t0, t1          ; [2|4|5|6] = 0x2456 (t0 - t1, updates CC) | MEM_ADDR = 0x0114  
BEQ  halt            ; [9|2|0E] = 0x920E (disp=+14: halt_byte=0x0132, not taken) | MEM_ADDR = 0x0116  
  
; branch 2: BR always taken  
BR   b2_land         ; [9|0|02] = 0x9002 (disp=+2: skips dead ADDI) | MEM_ADDR = 0x0118  
ADDI t2, t2, #-1      ; [1|6|6|F] = 0x166F (never executes) | MEM_ADDR = 0x011A
```

Steps

1. The Control Unit + Datapath structure had to be **divided in various stages**: IF, ID, EX, MEM.
2. The hazards were first all handled with stalls (RAW, load-use) or flushes (Branch).
3. Early Branch was implemented (ID) + predict-not-taken logic (IF) (the next instruction is the one immediately after the current PC)
4. RAW were treated with Data Forwarding/ *Bypassing*
5. Load-use hazards had to remain handled with the stall logic (given that there is no other way)

02

Stage Division

02.1

Overview

Separation of concerns

Each pipeline stage has a specific set of tasks it should perform. The implementation follows the design approach:

- ❖ IF: fetch the instruction
 - Set `next_pc` based on reset, branch taken and interrupt request
- ❖ ID: decode the instruction and perform the early-branch decision
 - Control Unit: performs the decoding and produces the required control signals for register handling and instruction identification
 - BDU: performs the branch-take decision
- ❖ EX: executes the instruction, using the ALU
- ❖ MEM: stores data in RAM/loads data from RAM when needed; writes value on register file (after completion, if applicable)

When happens what?

Flush: happens on a Branch-taken or an IRQ accept. Discards an instruction that has already entered the pipeline but must not execute.

be careful guys,
bubble and stall are
la meme chose

Stall: happens on Load-Use Hazards and memory waits

Bubble: inserted while upstream stages are frozen. The instruction behind the bubble is preserved in place and will advance on the next cycle. It fills an empty slot deliberately. Is inserted on Load-Use Hazards.

Forwarding: when an instruction in the execute stage uses a register(s) whose value is being changed on MEM stage, but hasn't been updated in the register file yet. Both operands may be forwarded.

REVER
ass: André :)

02.2

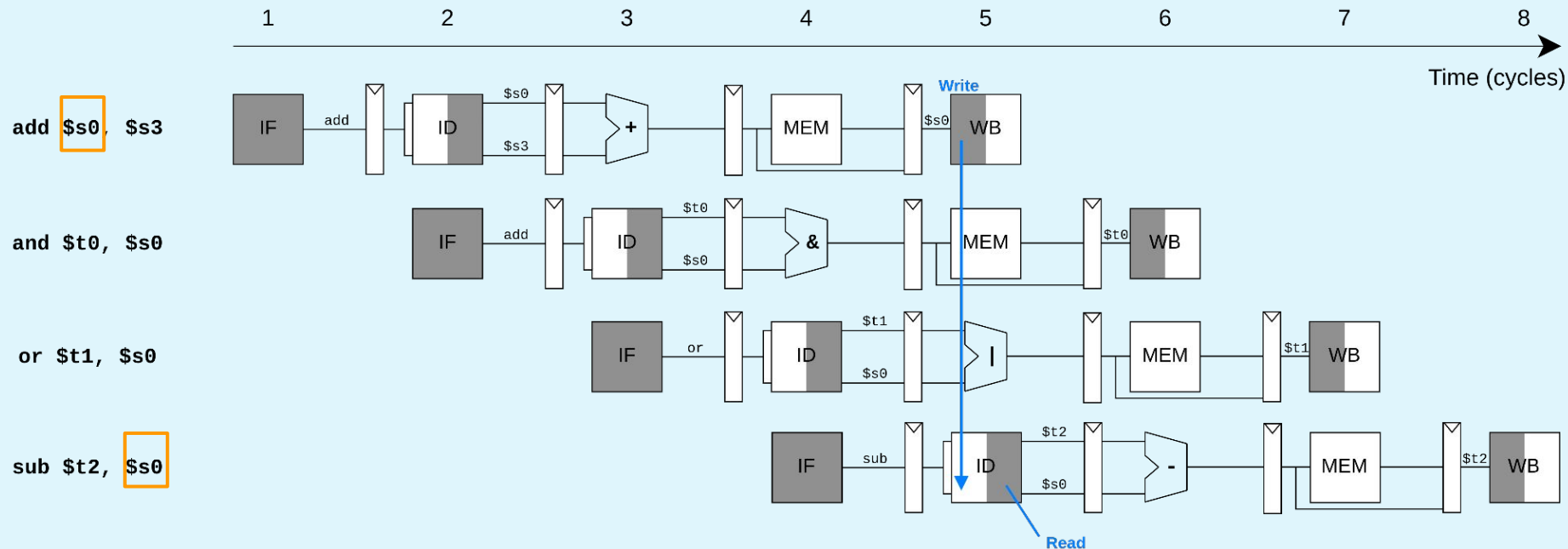
Memory Considerations

Register File

write-first-then-read

Scenario: When a register is being **updated on the WB stage** and in the **same clock cycle** it is being **read by the decode stage** (WB stage is used for simplicity purposes here – see next slide).

Without special care this leads to an hazard, where we need to stall the processor. A *write-first* Register File must be implemented.

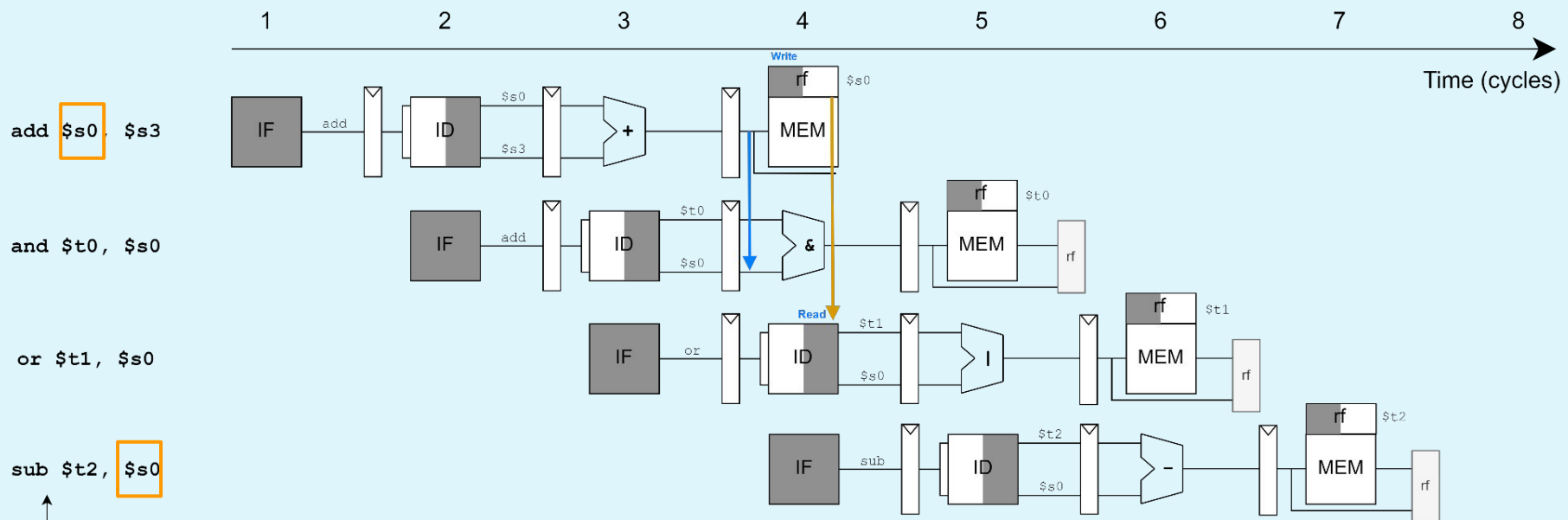


Register File

write-first-then-read

Scenario: When a register is being updated on the MEM stage and in the same clock cycle it is being read by the decode stage.

RF write on MEM stage



There is no Load instruction executing!
If there was, it would change the moment when RF
is written, but no memory stall is considered here.

This is the case of write-then-read of register file
(MEM writes and ID reads it in the same cycle)

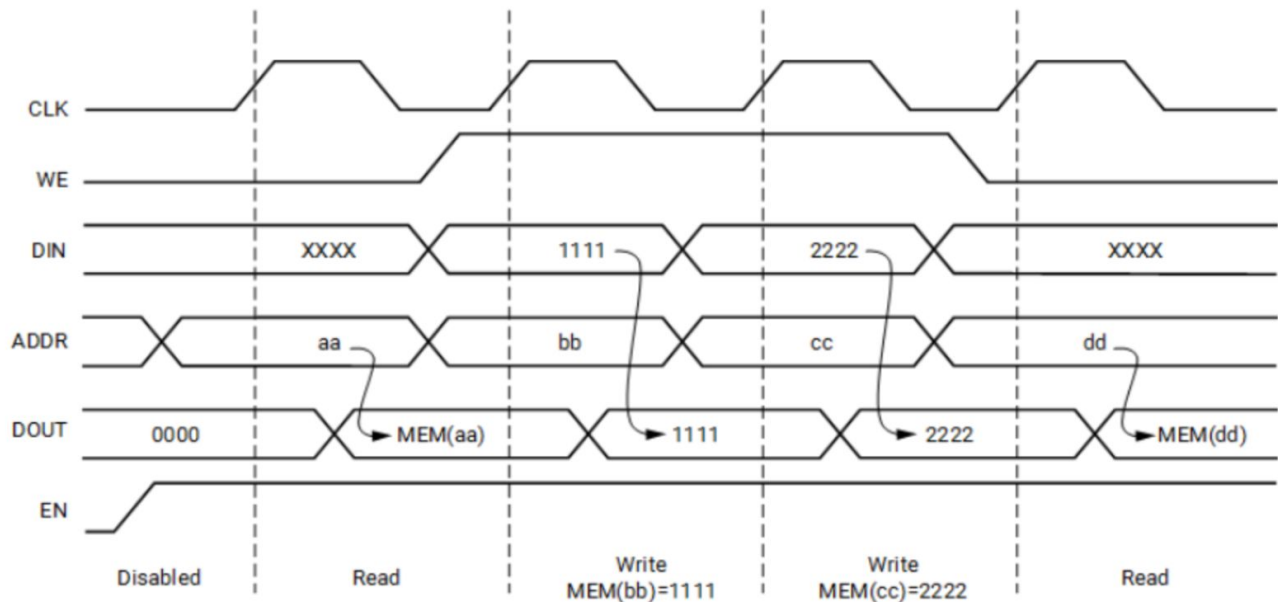
Register File

write-first-then-read

WRITE_FIRST or Transparent Mode

In WRITE_FIRST mode, the input data is simultaneously written into memory and stored in the data output (transparent write) as shown in the following figure. These waveforms correspond to latch mode when the optional output pipeline register is not used.

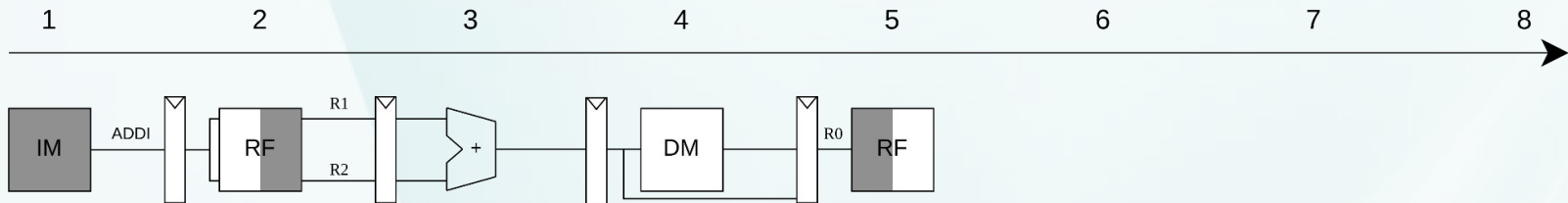
Figure: WRITE_FIRST Mode Waveforms



X21510-091218

Register File

write-first-then-read



```

/*****
* 2.2 Reset/Init and Write Path
*****/
initial begin
    for (_i = 0; _i < 16; _i = _i + 1) begin
        _mem[_i] = 16'h0000;
    end
end

always @(posedge i_clk) begin
    if (i_we && (i_wa != 4'b0000)) begin
        _mem[i_wa] <= i_wd;
    end
end

/*****
* 2.3 Read Path
*****/

assign o_ra = (i_we && (i_wa != 4'b0000) && (i_ra == i_wa)) ? i_wd : _mem[i_ra];
assign o_rb = (i_we && (i_wa != 4'b0000) && (i_rb == i_wa)) ? i_wd : _mem[i_rb];

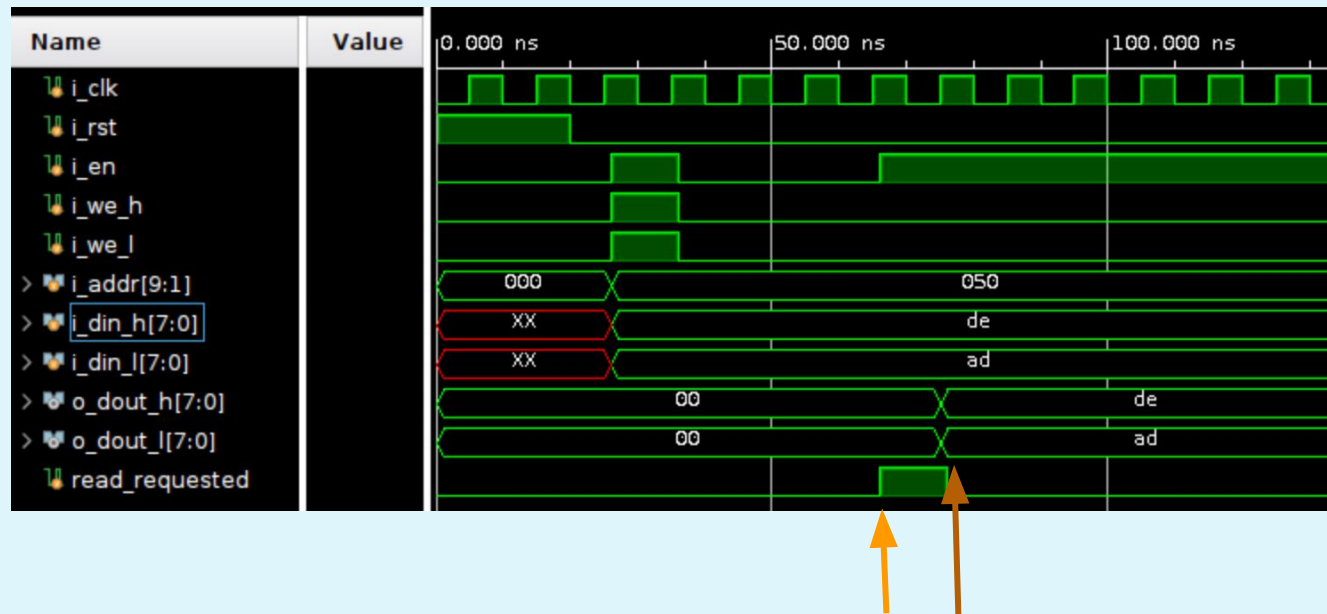
```

Same write
logic as
before

Bypass Logic

Allows for a read and
write on the same cycle

BROM & BRAM – registered (synchronous) read – **1 CLK cycle latency**



The value is outputted one cycle after it is requested

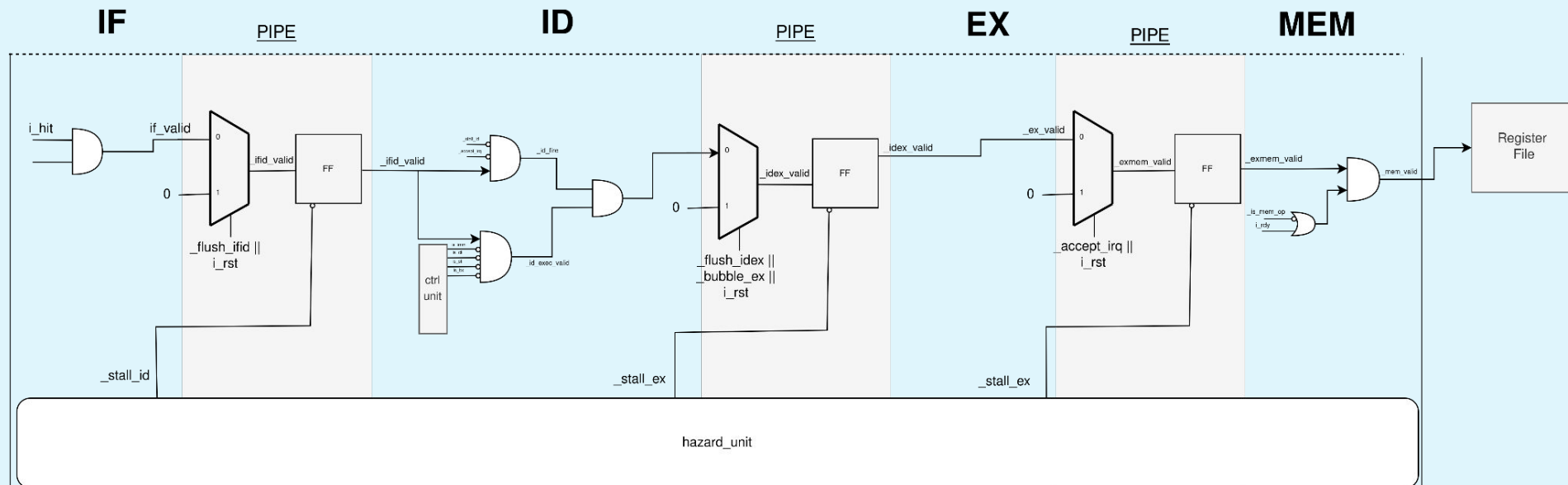
02.3

Stage Implementation & Hazard Handling

IF, ID (ctrl_unit+bdu), EX & MEM
Hazard & Forwarding Unit

The **valid** signal

The **valid** signal is the pipeline's **gating** mechanism — it travels with every instruction and determines whether that stage's outputs have any real effect on the processor state.



m_pipe_if_id

m_id_stage

Note: All FFs are positive-edge triggered on i_clk

```

/*****
 * 2.1 IF/ID Register
 *****/
always @(posedge i_clk) begin
    if (i_rst || i_flush) begin
        o_valid <= 1'b0;
        o_pc <= 16'h0000;
        o_insn <= `CPU_NOP_INSN;
    end else if (!i_stall) begin
        o_valid <= i_valid;
    end
end

```

```

// Exclude IMM/CLI/STI/BX from the EX pipeline: they are handled entirely in ID
assign o_exec_valid = i_valid & ~(is_imm | is_cli | is_sti | is_bx);

```

m_cpu

```

// Register file writeback - driven directly from MEM stage
assign _wb_rf_we = _mem_valid & _mem_rf_we;

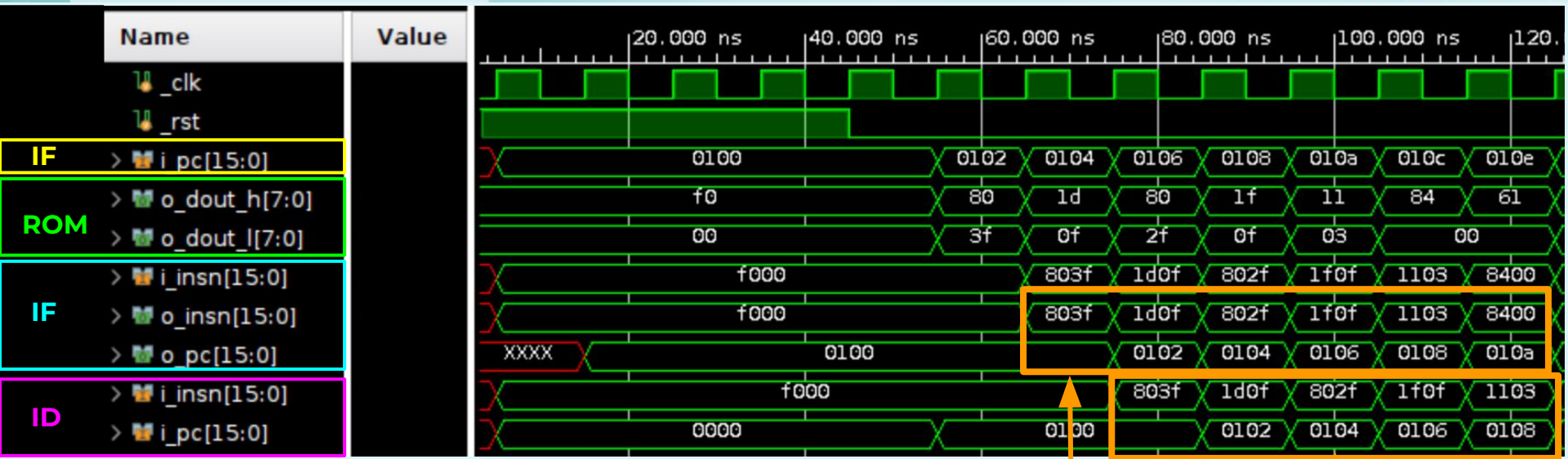
```

```
*****  
* SECTION 2. IMPLEMENTATION  
*****  
  
// Instruction memory is enabled only when there is a hit and no stall.  
assign o_insn_ce = i_hit & ~i_stall;  
  
// Output is valid only when: instruction memory has a hit AND there are  
// no outstanding flush bubbles still being drained.  
assign o_valid = i_hit & (_flush_bubble == 2'd0);  
  
// Pass the instruction word with validation  
// If insn is not valid, a bubble (NOP) is inserted  
assign o_insn = (_flush_bubble == 2'd0) ? i_insn : `CPU_NOP_INSN;  
  
always @(posedge i_clk) begin  
    if (i_rst) begin  
        // On reset, seed the entire PC pipeline with the reset address  
        // so there is no spurious old-PC instruction in flight.  
        _pc_d1 <= i_pc;  
        o_pc <= i_pc;  
        _flush_bubble <= 2'd0;  
    end else if (i_flush) begin  
        // Redirect: load the flush target into all pipeline stages immediately,  
        // and arm the 1-cycle bubble counter.  
        _pc_d1 <= i_flush_pc;  
        o_pc <= i_flush_pc;  
        _flush_bubble <= 2'd2;  
    end else begin  
        if (i_hit & ~i_stall) begin  
            // Normal advance: shift the PC pipeline forward and  
            // decrement the bubble counter if it is non-zero.  
            _pc_d1 <= i_pc;  
            o_pc <= _pc_d1;  
            if (_flush_bubble != 2'd0) begin  
                _flush_bubble <= _flush_bubble - 2'd1;  
            end  
        end  
    end  
end  
  
// If stalled (i_stall asserted) or no hit: hold everything frozen.  
end  
end
```

When it is required to insert a bubble, due to the flush on branch or interrupt, bubbles are inserted.

Next slide

PC logic



SOC.V

brom.v

ROM

1 cycle
latency

```
*****  
* 2.2 Instruction Fetch Latch  
*****  
always @(posedge i_clk) begin  
    if (i_rst | _imem_invalid) begin  
        _insn_q <= _default_nop;  
    end else if (_insn_ce) begin  
        _insn_q <= _imem_dout;  
    end  
end
```

1 cycle
latency



```
end else begin  
    if (i_hit & ~i_stall) begin  
        // Normal advance: shift the PC pipeline forward and  
        // decrement the bubble counter if it is non-zero.  
        _pc_d1 <= i_pc;  
        o_pc <= _pc_d1;  
        if (_flush_bubble != 1'd0) begin  
            _flush_bubble <= _flush_bubble - 1'd1;  
        end  
    end  
end  
// If stalled (i_stall asserted) or no hit: hold everything frozen.
```

m_if.v

Compensate the latency
(i_pc is the used variable to fetch the isns
from ROM)

Instantiates the **Control Unit** which decodes the instruction, setting relevant signals for instruction identification, register usage, RF WE, interrupt admissible (...)

The control unit generates important control signals which will travel through the pipeline, such as o_rf_we, o_reads_rd, o_reads_rs and o_writes_rd.

```
/* SECTION 2. IMPLEMENTATION
*****

// Instantiate the combinational instruction decoder
ctrl_unit u_ctrl_dec (
    .i_insn(i_insn),
    .o_rd(rd),
    .o_rs(rs),
    .o_fn(fn),
    .o_imm(imm),
```

(...)

```
/* *****
 * 2.2 Side-Effect and Hazard Qualifiers
 * *****
/

// Instructions that must prevent an IRQ from being accepted while in the ID stage:
// - IMM prefix: the following instruction is paired with this one; an IRQ between
//   them would corrupt the immediate state.
// - ADC/SBC/CMP: these read the carry flag; an IRQ between a cc-update and
//   the carry-consumer would corrupt the carry value seen by the instruction.
assign _interlocked_insns = _is_imm | (_is_alu & (_is_adc | _is_sbc | _is_cmp));

// Hazard-detection register read qualifiers
assign o_reads_rd = _is_rr | _is_ri | _is_sw | _is_sb;
assign o_reads_rs = _is_rr | _is_addi | _is_lw | _is_lb | _is_sw | _is_sb | _is_jal | _is_setcc;
// Writeback qualifier: CMP does not write a result despite being an ALU op
assign o_writes_rd = (_is_alu & ~_is_cmp) | _is_addi | _is_lb | _is_lw | _is_jal | _is_getcc;
assign o_is_load = _is_lw | _is_lb;
assign o_is_store = _is_sw | _is_sb;

assign o_uses_cc = _is_bx;
assign o_uses_carry = _is_alu & (_is_adc | _is_sbc);
// An instruction updates CCs if it performs arithmetic (sum/cmp group) or ADDI or SETCC
assign o_updates_cc = ((_is_alu & (_is_sum | _is_cmp)) | _is_addi | _is_setcc);
assign o_irq_interlock = _interlocked_insns;
```

Instantiates the **Branch Decision Unit** which is responsible for analysing the branches, via CC and deciding whether the branch will or won't be taken.

```
// Branch Decision Unit: evaluates the condition-code field against the
// current architectural flags to decide if a BX branch is taken.
bdu u_bdu (
    .i_cond(_cond),
    .i_ccz(i_ccz),
    .i_ccn(i_ccn),
    .i_ccc(i_ccc),
    .i_ccv(i_ccv),
    .o_take(_bdu_take)
);
```

```
/* *****
 * SECTION 2. IMPLEMENTATION
 * ***** */

always @(*) begin
    case (i_cond & 4'b1110)
        `BR_BR:    _t = 1'b1;
        `BR_BEQ:   _t = i_ccz;
        `BR_BC:    _t = i_ccc;
        `BR_BV:    _t = i_ccv;
        `BR_BLT:   _t = i_ccn ^ i_ccv;
        `BR_BLE:   _t = (i_ccn ^ i_ccv) | i_ccz;
        `BR_BLTU:  _t = ~i_ccz & ~i_ccc;
        `BR_BLEU:  _t = i_ccz | ~i_ccc;
        default:   _t = 1'b0;
    endcase
end

assign o_take = i_cond[0] ? ~_t : _t; // invert condition for odd cond values (e.g.
```

```
// ---- Immediate construction ----

// Word-offset instructions (LW, SW, JAL) encode the address offset in units of 2 bytes.
// Because all memory accesses are naturally aligned, bit 0 of the immediate is
// repurposed: it contributes to bit 1 of the byte address (i.e., the offset is × 2).
assign _word_off = _is_lw | _is_sw | _is_jal;

// Sign-extension of the 4-bit immediate for ADDI and ALU:
// bit 3 of the immediate is the sign bit.
assign _sxi = (_is_addi | _is_alu) & _imm[3];
assign _sx11 = {11{ _sxi }}; // Replicate sign bit across upper 11 bits

// Reconstruct bits [4] and [0] of the effective immediate:
// bit 4 = sign-extend OR (word-aligned AND original bit 0 in the offset role)
// bit 0 = 0 for word-offset instructions (offset is already × 2); otherwise = _imm[0]
assign _i_4 = _sxi | (_word_off & _imm[0]);
assign _i_0 = (~_word_off) & _imm[0];

// Final imm16:
// - If preceded by an IMM prefix: concatenate the saved 12-bit payload with
//   the current 4-bit immediate field → full 16-bit literal.
// - Otherwise: build from sign/zero-extended 4-bit immediate.
assign imm16 = i_imm_pre_state ? {i_i12_pre_state, _imm}
                               : { _sx11, _i_4, _imm[3:1], _i_0 };
```

Immediate Logic, same as in the GR0040

Branch target computation, same as in the GR0040

Evaluates `o_branch_take` based on `bdu` output

```
// ---- Branch displacement ----
// Sign-extend the 8-bit displacement, then shift left by 1 (×2, word-addressing).
assign _sxd7 = {7{ _disp[7] }};
assign _sxd16 = { _sxd7, _disp, 1'b0 }; // {sign×7, disp[7:0], 0} = signed offset in bytes

// ---- Branch decision ----
// Branch is taken if: instruction is valid AND (it's a JAL unconditional, OR
// it's a BX with the condition satisfied by the current flags).
assign o_branch_take = i_valid & (_is_jal | (_is_bx & _bdu_take));
// Branch target:
// JAL → register indirect: Rs + imm16
// BX → PC-relative: PC + sign_extend(disp) × 2
assign o_branch_target = _is_jal ? (_imm16 + i_rs_data) : (i_pc + _sxd16);

// ---- Output assignments ----
assign o_valid = i_valid;
```

Assign valid signals.

`o_exec_valid` is extremely important since some instructions get completely resolved on the decode stage, so their execution in the downstream stage is irrelevant

```
// Exclude IMM/CLI/STI/BX from the EX pipeline: they are handled entirely in ID
assign o_exec_valid = i_valid & ~(i_is_imm | i_is_cli | i_is_sti | i_is_bx);
```

```
// muxes for the forwarding
assign _rd_fwd = (i_forward_a == 1'b1) ? i_exmem_wb_data : i_rd_data;
assign _rs_fwd = (i_forward_b == 1'b1) ? i_exmem_wb_data : i_rs_data;
```

```
// Operand mux for RI vs RR:
// RI: Rd is the destination AND the left source; Rs is unused.
//   _src = Rd_data (right operand for ALU)
//   _a   = imm16   (left operand, the immediate)
// RR: Rd is left, Rs is right.
//   _src = Rs_data
//   _a   = Rd_data
assign _src = i_is_ri ? _rd_fwd : _rs_fwd;
assign _a   = i_is_rr ? _rd_fwd : i_imm16;
assign _b   = _src;

// Add/subtract control:
// Subtraction operations (SUB, SBC, CMP) use _add=0 to signal the ALU
// to negate the B operand (two's-complement adder with inverted carry-in).
assign _add = ~(i_is_alu & (i_is_sub | i_is_sbc | i_is_cmp));

// Carry-in:
// Addition: ci = _c (propagate carry for ADC, or 0-equivalent for ADD via _c init)
// Subtraction: ci = ~_c (borrow = ~carry convention)
assign _ci = _add ? i_c : ~i_c;

alu u_alu (
    .i_is_add(_add),
    .i_a(_a),
    .i_b(_b),
    .i_ci(_ci),
    .i_is_xor(i_is_xor),
    .i_is_sra(i_is_sra),
    .o_sum(_sum),
    .o_log(_log),
    .o_sr(_sr),
    .o_co(_c_w), // Raw carry-out from adder
    .o_x(_x)
);
```

Analyse if **forwarding** is required. If it is, forward the data from MEM→EX. Otherwise, use the input operand.

Execution Logic, same as in the GR0040

It also performs:

- condition code update (psw_vector)
- carry update (for adc and sbc)

(...)

Handles writing the data to memory if required.
Produces the resource stall trigger signal when there is a memory operation and memory is not ready.

```
/* 2.1 DM access */
// Any of the four memory strobes means this is a memory operation
assign _is_mem_op = i_lw | i_lb | i_sw | i_sb;

// Pipeline stall: instruction is valid, it's a memory op, and memory isn't ready yet
assign o_mem_wait = i_valid & _is_mem_op & ~i_rdy;

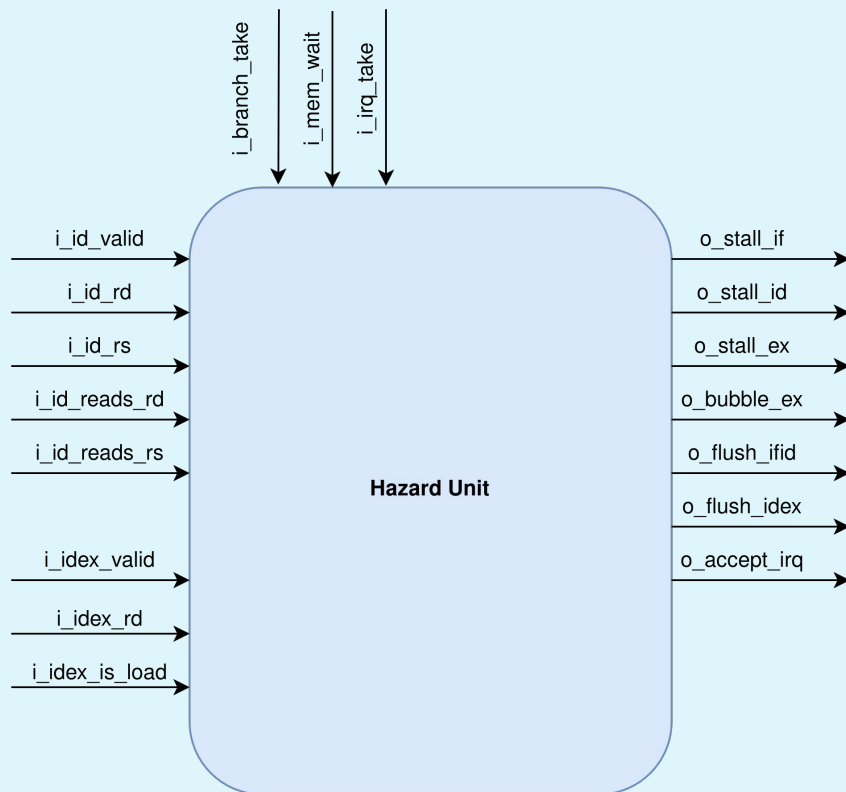
// Memory complete: either there's no memory op, or the memory op has finished
assign o_mem_complete = i_valid & (~_is_mem_op | i_rdy);
```

Computes the required signals that feedback to the pipeline in order to **stall** it if there is a memory operation and memory is not ready yet (o_mem_wait)

i_rdy is computed in the soc module and tracks if there is a load instruction ongoing. If so, memory is set to not ready.

Hazard Unit Inputs

The Hazard Unit (including the Forwarding Unit) is responsible for generating the control signals required to stall and flush the pipeline when required.



```
module hazard_unit(
    // ---- Instruction in ID ----
    input wire i_id_valid,
    input wire [3:0] i_id_rd,           // Rd field of instruction in ID
    input wire [3:0] i_id_rs,           // Rs field
    input wire i_id_reads_rd,          // Instruction reads Rd as a source
    input wire i_id_reads_rs,          // Instruction reads Rs as a source

    // ---- External control events ----
    input wire i_branch_take,          // A branch/jump was committed in ID this cycle
    input wire i_mem_wait,             // MEM stage is waiting for data memory
    input wire i_irq_take,             // An interrupt request is pending (one-shot)

    // ---- ID/EX stage state (instruction in EX) ----
    input wire i_idex_valid,
    input wire [3:0] i_idex_rd,        // Destination register of EX instruction
    input wire i_idex_is_load,         // EX instruction is a load (extra latency)

    // ---- Control outputs ----
    output wire o_stall_if,
    output wire o_stall_id,
    output wire o_stall_ex,
    output wire o_bubble_ex,
    output wire o_flush_ifid,
    output wire o_flush_idex,
    output wire o_accept_irq
);
```

Hazard Unit

Control Outputs

```
*****
* 2.2 Control Outputs
*****/

// An IRQ can only be accepted when:
// - The interrupt line is asserted (one-shot pulse), AND
// - The MEM stage is not stalling (we don't want to disturb an in-flight
//   memory operation by also trying to save PC into R14 simultaneously)
assign _accept_irq = i_irq_take & ~i_mem_wait;

// IF stalls when data memory is busy OR there is a decode hazard.
// Both cases freeze the PC so the same instruction is re-fetched.
assign o_stall_if = i_mem_wait | _decode_hazard;

// ID stalls for the same reasons as IF (they are always stalled together).
assign o_stall_id = i_mem_wait | _decode_hazard;

// EX stalls only during a MEM wait (the pipeline above MEM freezes).
assign o_stall_ex = i_mem_wait;

// Upon Load-use hazard detection => Load insn is in EX and previous insn in ID
// The goal is not to insert a bubble on the ID/EX register that propagates along the pipeline.
assign o_bubble_ex = _decode_hazard & ~i_mem_wait;

// Flush IF/ID on branch commit or IRQ accept (both redirect the PC).
assign o_flush_ifid = i_branch_take | _accept_irq;

// Flush ID/EX on IRQ accept to squash the instruction that was about to enter EX.
// (A branch commit does NOT need to flush ID/EX because the instruction in ID
// is the branch itself - it has already been handled.)
assign o_flush_idex = _accept_irq;

assign o_accept_irq = _accept_irq;
```

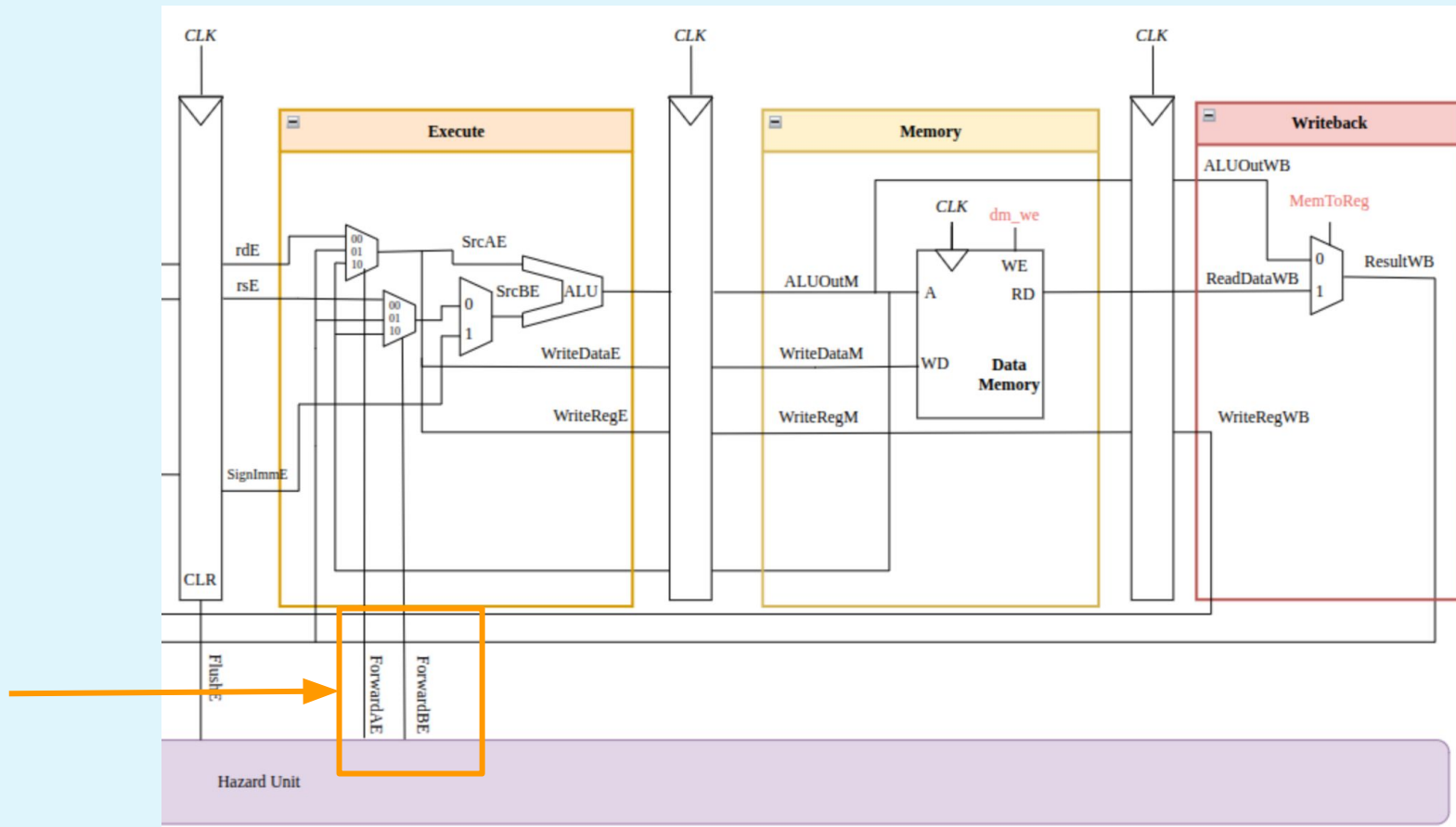
- ❖ **i_irq_take** comes from cpu module
- ❖ **i_mem_wait** comes from MEM stage and is connected to BRAM (instantiated in the soc module and passed to the cpu module)
 - The soc module passes the variable **_rdy** to the cpu module;
`_rdy = _is_io ? _io_rdy : _mem_rdy;`
 - When writing to MMIO, **_io_rdy** is an echo variable of the peripheral (the only requirement is that the selected peripheral exists)
 - When **writing to BRAM**, **_mem_rdy** is tracked in the soc module (not inside BRAM) and it **depends purely** on the **LOAD** instruction
`_mem_rdy = ~((_lw | _lb) & ~_loaded);`
- ❖ **_decode_hazard** depends on a **load-use hazard**, which depends on
 1. register matching in EX and ID,
 2. current instruction is a load,
 3. instruction in ID is valid (i.e., o_valid propagated from IF is 1)
 4. rd is not 0
- ❖ **o_bubble_ex** inserts a bubble in ID/EX, the moment a load_use hazard occurs

Note: When addressing the Hazards, the stall and flush signals generation will be addressed in more detail.

Hazard Unit Forwarding

Issue: Execute stage requires a register (rs or rd) which is being updated and is on MEM or WB.

Solution: Forward that value from MEM to EX or from WB to EX (MEM has higher priority)



Hazard Unit Forwarding Unit

The Forwarding Unit inputs consist on control signals to know whether a register is being written on a stage.

```
module m_forwarding (  
    // Execute (EX) Stage Inputs  
    input wire [3:0] i_rdE,          // Source register 1 (Operand A)  
    input wire [3:0] i_rsE,          // Source register 2 (Operand B)  
  
    // Memory (MEM) Stage Inputs  
    input wire [3:0] i_WriteRegM,    // Destination register in MEM stage  
    input wire      i_RegWriteM,     // Write Enable signal in MEM stage  
  
    // Verify if the instruction on MEM is valid  
    input wire i_validM,  
  
    // Forwarding Multiplexer Control Outputs  
    output wire o_ForwardAE,         // Selector for ALU input A  
    output wire o_ForwardBE         // Selector for ALU input B  
);
```

Hazard Unit Forwarding Unit

(rsE != 0) AND (rsE == WriteRegM) AND RegWriteM	Forward AE	
0	0	From Register File (Normal path, no hazard)
1	1	From MEM/WB register (Forwarding from WB)

```
assign o_ForwardAE = (i_rdE != 4'h0) && (i_rdE == i_WriteRegM) && i_RegWriteM && i_validM;  
assign o_ForwardBE = (i_rsE != 4'h0) && (i_rsE == i_WriteRegM) && i_RegWriteM && i_validM;
```

02.4

Pipes

(intermediate registers)

pipe_IFID, pipe_IDEX & pipe_EXMEM

Intermediate Registers

The intermediate registers(pipes) placed in between the stages store the data transmitted between stages. They are able of **stalling** or **flushing** the data, according to the respective control signals computed in the Hazard Unit.

When stalling the previous values on the registers are maintained. When flushing, all values are zeroed.

```
/* *****  
 * 2.1 ID/EX Register  
 * ***** */  
  
always @(posedge i_clk) begin  
    if (i_rst || i_flush || i_bubble) begin  
        // Clear to NOP: zero all control signals so EX sees a harmless bubble  
        o_valid      <= 1'b0;  
        o_pc         <= 16'h0000;  
        o_rd         <= 4'h0;  
        o_rs         <= 4'h0;  
(...)   
    end else if (!i_stall) begin  
        // Normal advance: capture the full decode bundle  
        o_valid      <= i_valid;  
        o_pc         <= i_pc;  
        o_rd         <= i_rd;  
        o_rs         <= i_rs;  
(...)   
    end  
end
```

m_pipe_id_ex.v

02.5

Top Module Implementation

SoC & CPU (pipes + pc_next)

SoC module

The SoC module instantiates all the modules (cpu, peripheral, bram and brom) and **remained unchanged** as expected.

03

Hazards

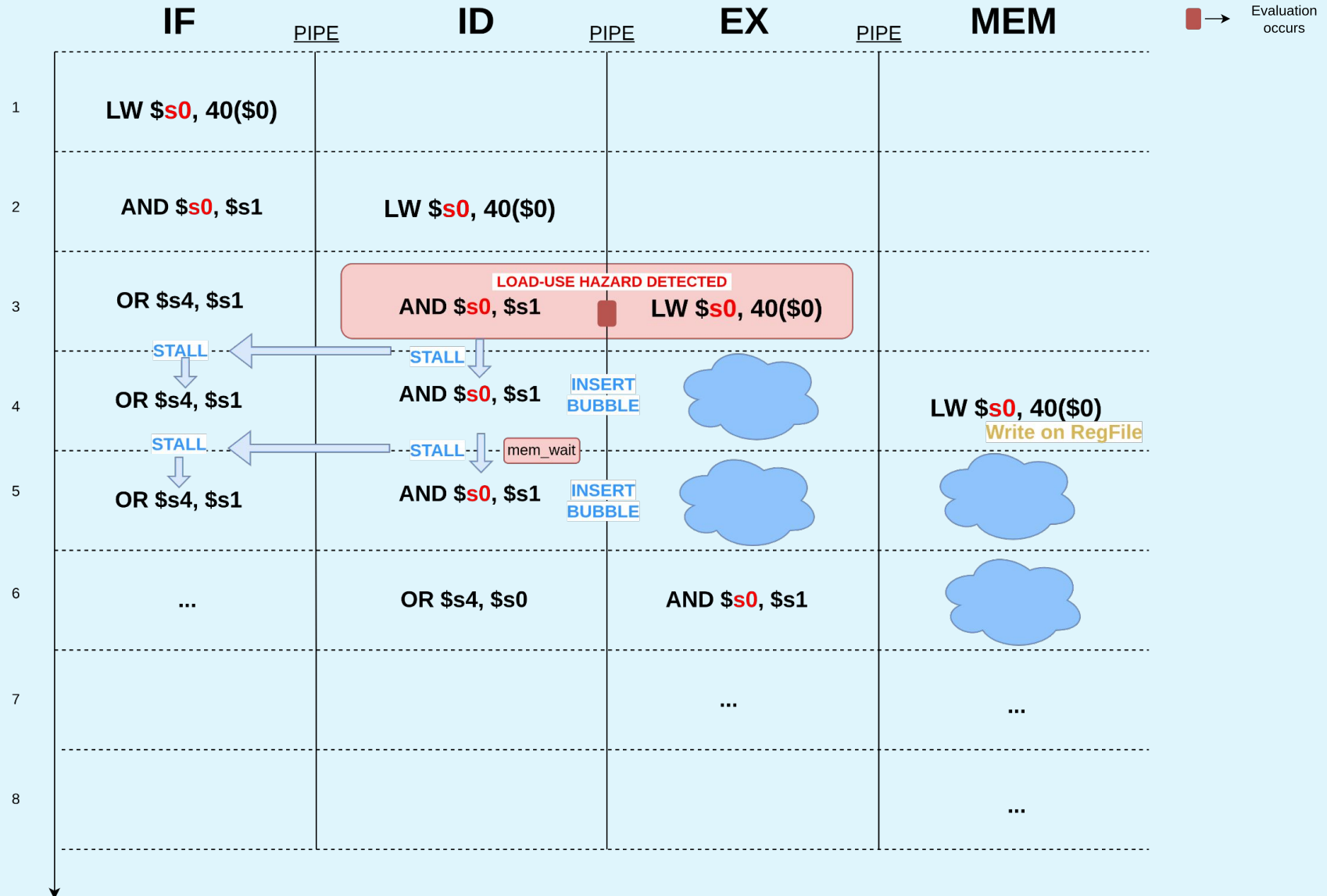
03.1

Load-Use Interlock

Load-Use Interlock

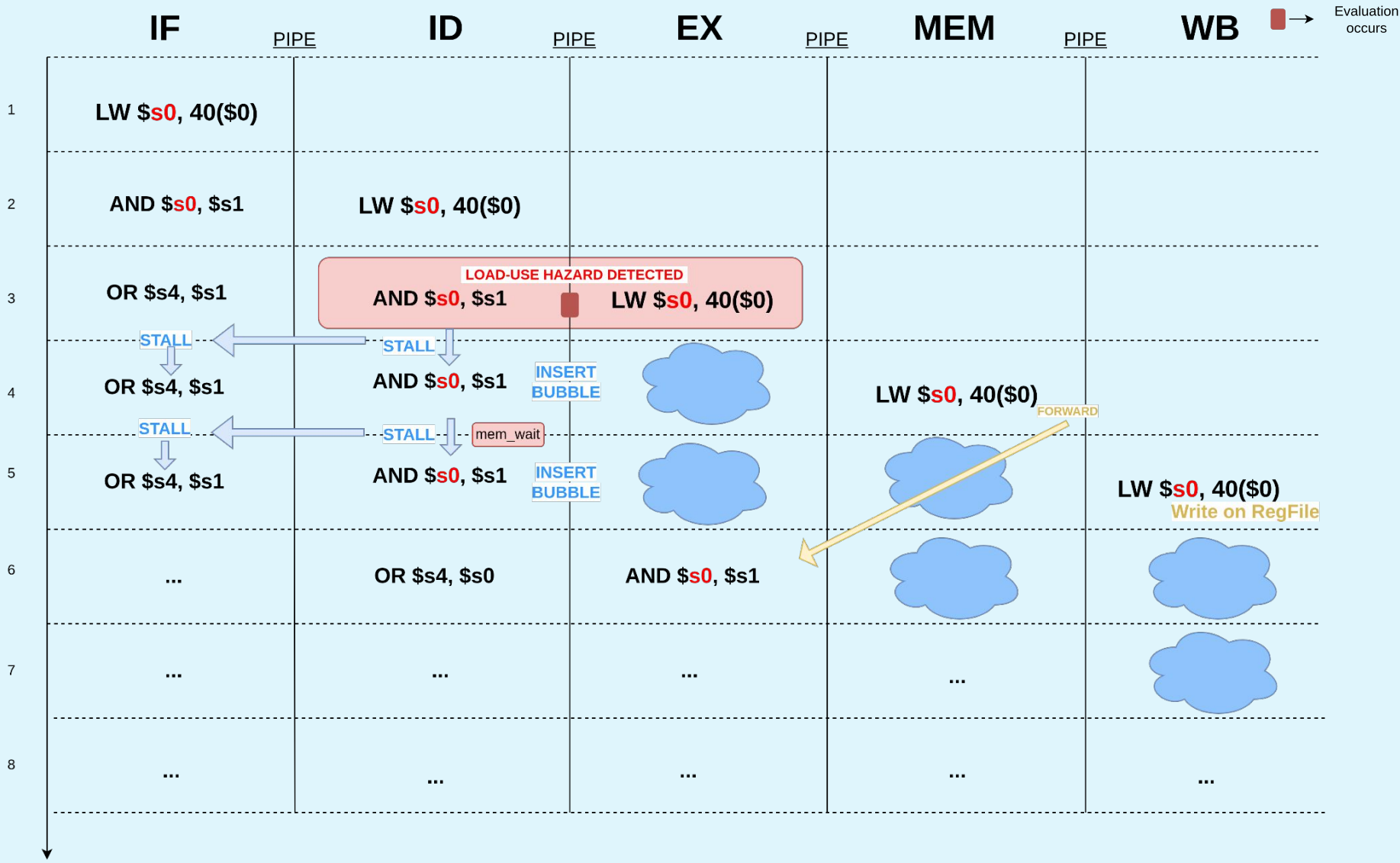


CENTROALGORITMI



Load-Use Hazard

Compare with **WB** stage
inclusion



Hazard Unit

Load-Use Interlock

Load-use Hazards are evaluated between the ID and EX stages. Upon a load instruction, if there is a register match between the operands on the following instruction and the destination operand on the load instruction, a load-use stall is required.

```
// Register address match: ID instruction reads a register that the EX stage
// is about to write. Used exclusively for load-use hazard detection.
assign _match_idx = (i_id_reads_rd & (i_id_rd == i_idex_rd)) | (i_id_reads_rs & (i_id_rs == i_idex_rd));

// Load-use hazard: the instruction immediately following a load reads the loaded
// register. The load result is only available after MEM, so an extra stall is
// needed. R0 destination excluded - writes to R0 are discarded.
// Ignore load-use hazards on rd=r0, since r0 is architecturally constant zero.
assign _load_use_hazard = i_id_valid & i_idex_valid & i_idex_is_load &
    (i_idex_rd != 4'h0) & _match_idx;
```

The load-use stall implies that:

1. IF and ID are stalled (o_stall_if and o_stall_id)
2. ID/EX pipe must insert a bubble (indicate that passed data is not valid) – o_bubble_ex
3. EX shall not update any conditions – i_valid on EX depends on _id_fire:

```
// _id_fire: the ID stage commits an instruction this cycle.
// Requires a valid instruction in IF/ID, no stall, and no concurrent IRQ accept.
assign _id_fire = _ifid_valid & ~_stall_id & ~_accept_irq;
```

Hazard Unit

Load-Use Interlock

Given the **memory latency** (synchronous read memory) the load-use hazard will stall 2 clock cycles, inserting 2 bubbles (zeroing out all Execute stage control signals during a decode stage stall).

Stall 1 is due to Load-Use Hazard – **_load_use_hazard**

Stall 2 is due to Memory Latency – **i_mem_wait**

Load-Use Interlock Validation – Test Scenarios

Test Code

load-use_stall_00.asm

```
; =====  
; TEST 1: load-use with 0 gap – 2 stalls expected  
; AND enters ID while LW is still in MEM (mem_wait cycle 1)  
; pipeline stalls AND twice before t0 is available via WB→EX forward  
; expected: t1 = 0xFF & 7 = 7  
; =====  
; =====  
; TEST 2: load-use with 1 gap – 1 stall expected  
; 1 independent instruction absorbs 1 stall cycle, 1 stall still needed  
; expected: t1 = 7 & 7 = 7  
; =====  
; =====  
; TEST 3: load-use with 2 gaps – 1 stall expected  
; 2 independent instructions fully hide the 2-cycle memory latency  
; expected: t1 = 0xFF & 7 = 7  
; =====  
; =====  
; TEST 4: back-to-back loads both used immediately  
; each LW→use pair causes 2 stalls independently  
; expected: t2 = 7 + 3 = 10  
; =====  
; =====  
; TEST 5: load-use on store data operand (0 gap)  
; SW uses t0 as the data to write – hazard unit must also stall for this case  
; expected: MEM[sp+2] = 7  
; =====
```

Load-Use Interlock Validation



Test Code load-use_stall_00.asm

```

reset:
    LI    sp, #STACK_TOP        ; => IMM #0x03F      [8|03F] = 0x803F (prefix: upper 12 bits of STACK_TOP=0x03FF) | MEM_ADDR = 0x0100
                                ; => ADDI sp,zero,#0xF    [1|D|0|F] = 0x1D0F (sp = 0x03FF with prefix) | MEM_ADDR = 0x0102

main:
    ; Deactivate TimerH int_en
    IMM   #0x408                ; [8|408] = 0x8408 (prefix: upper 12 bits of MMIO word-address 0x8100) | MEM_ADDR = 0x0104
    SW    r0, r0, #0            ; [6|0|0|0] = 0x6000 (MEM[0+0x8100] = r0 = 0, using prefix) | MEM_ADDR = 0x0106

    ; Setup: store known values into memory for later loads
    ; MEM[sp+0] = 7, MEM[sp+1] = 3
    LI    t0, #7                ; => IMM #0x000      [8|000] = 0x8000 (prefix: 7>>4 = 0) | MEM_ADDR = 0x0108
                                ; => ADDI t0,zero,#0x7    [1|4|0|7] = 0x1407 (t0 = 7) | MEM_ADDR = 0x010A
    SW    t0, sp, #0            ; [6|4|D|0] = 0x64D0 (MEM[sp+0] = 7) | MEM_ADDR = 0x010C

    LI    t0, #3                ; => IMM #0x000      [8|000] = 0x8000 (prefix: 3>>4 = 0) | MEM_ADDR = 0x010E
                                ; => ADDI t0,zero,#0x3    [1|4|0|3] = 0x1403 (t0 = 3) | MEM_ADDR = 0x0110
    SW    t0, sp, #1            ; [6|4|D|1] = 0x64D1 (MEM[sp+1] = 3) | MEM_ADDR = 0x0112

    ; t1 = 0xFF (mask - AND with any value ≤ 0xFF gives that value directly)
    LI    t1, #0xFF             ; => IMM #0x00F      [8|00F] = 0x800F (prefix: 0xFF>>4 = 0xF) | MEM_ADDR = 0x0114
                                ; => ADDI t1,zero,#0xF    [1|5|0|F] = 0x150F (t1 = 0x00FF) | MEM_ADDR = 0x0116

    ; =====
    ; TEST 1: load-use with 0 gap - 2 stalls expected
    ; AND enters ID while LW is still in MEM (mem_wait cycle 1)
    ; pipeline stalls AND twice before t0 is available via WB→EX forward
    ; expected: t1 = 0xFF & 7 = 7
    ; =====
    LW    t0, sp, #0            ; [4|4|D|0] = 0x44D0 (t0 = MEM[sp+0] = 7) | MEM_ADDR = 0x0118
    AND   t1, t0                ; [2|5|4|2] = 0x2542 (t1 = 0xFF & 7 = 7 - load-use: 0 gap → 2 stalls) | MEM_ADDR = 0x011A

    ; =====
    ; TEST 2: load-use with 1 gap - 1 stall expected
    ; 1 independent instruction absorbs 1 stall cycle, 1 stall still needed
    ; expected: t1 = 7 & 7 = 7
    ; =====
    LI    t1, #0xFF             ; => IMM #0x00F      [8|00F] = 0x800F (prefix: 0xFF>>4 = 0xF) | MEM_ADDR = 0x011C
                                ; => ADDI t1,zero,#0xF    [1|5|0|F] = 0x150F (t1 = 0x00FF) | MEM_ADDR = 0x011E
    LW    t0, sp, #0            ; [4|4|D|0] = 0x44D0 (t0 = MEM[sp+0] = 7) | MEM_ADDR = 0x0120
    ADDI   s0, s0, #0            ; [1|8|8|0] = 0x1880 (gap: independent, absorbs 1 stall cycle) | MEM_ADDR = 0x0122
    AND   t1, t0                ; [2|5|4|2] = 0x2542 (t1 = 0xFF & 7 = 7 - load-use: 1 gap → 1 stall) | MEM_ADDR = 0x0124

```

(...) (continues)

Test Code **load-use_stall_00.asm**: Test **1, 2 and 3** pseudo-code analysis.

```
t0=0x07  
MEM[sp+0]=t0 //⇔MEM[sp+0]=0x07  
t0=0x03  
MEM[sp+0]=t0 //⇔MEM[sp+0]=0x03
```

RAW Hazard → **FORWARDING**: WB→EX

```
t0=0x03  
MEM[sp+0]=t0 //⇔MEM[sp+0]=0x03
```

RAW Hazard → **FORWARDING**: WB→EX

```
t1=0xFF  
t0=MEM[sp+0] //⇔t0=0x07  
t1=t1&t0 // =0x07
```

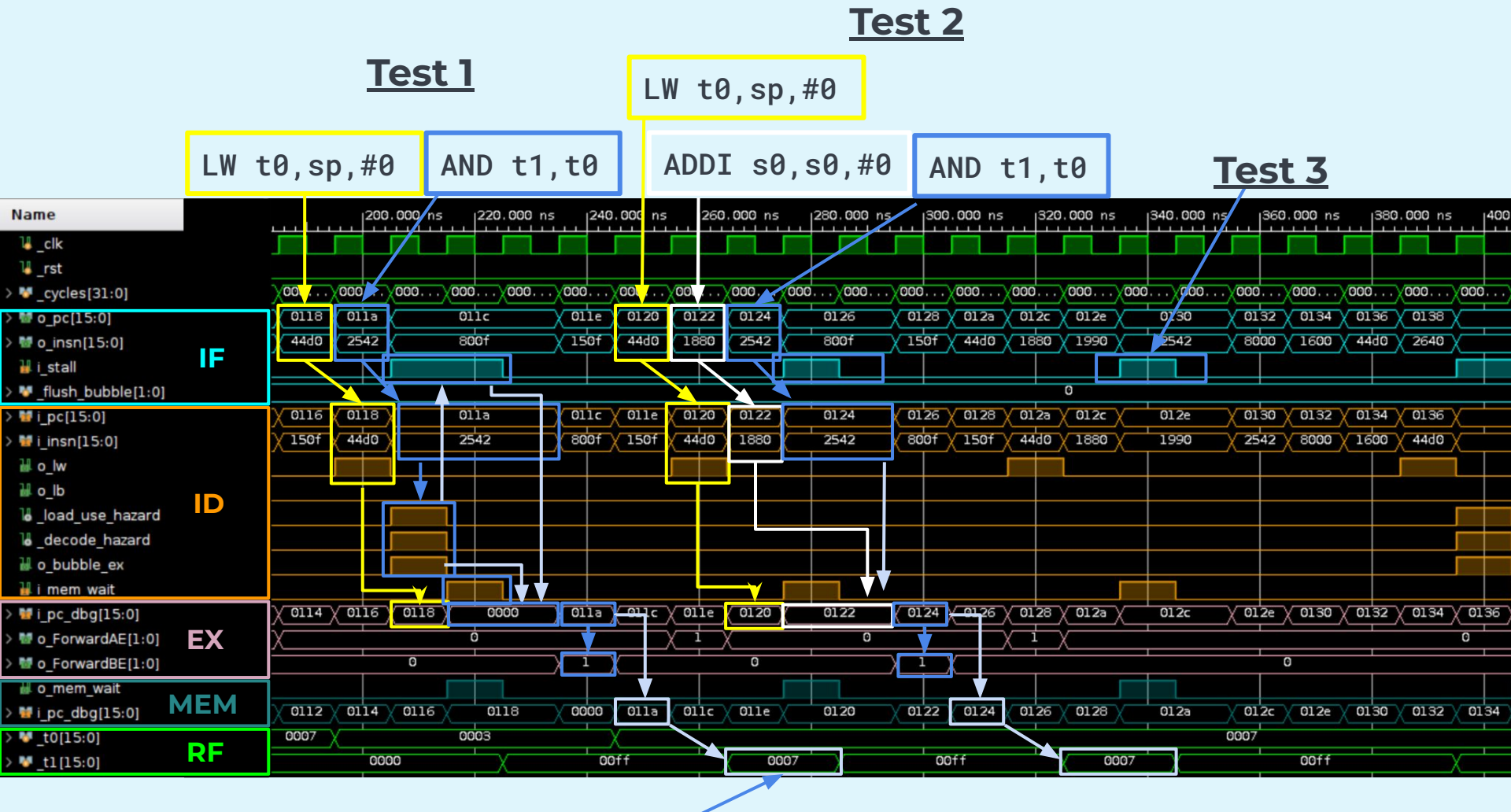
Load-Use Hazard with **gap 0** between
producer and consumer instruction
→ **2 stalls expected**, followed by
FORWARDING WB→EX

```
t1=0xFF  
t0=MEM[sp+0] //⇔t0=0x07  
s0=s0+0  
t1=t1&t0 // =0x07
```

Gap 1 between producer and
consumer instruction
→ **1 stall expected**, followed by
FORWARDING WB→EX

```
t1=0xFF  
t0=MEM[sp+0] //⇔t0=0x07  
s0=s0+0  
s0=s0+0  
t1=t1&t0 // =0x07
```

Gap 2 between producer and
consumer instruction
→ **1 stall expected**, followed by
FORWARDING WB→EX



03.2

Data Hazards (RAW)

Forwarding

Validation – Test Scenarios

Test Code
forwarding_00.asm

```
; =====  
; TEST 1: MEM→EX forwarding (distance 1)  
; t0 produced in EX, consumed next cycle when t0 is in MEM  
; without forwarding: t0 still in EX → regfile read returns stale value  
; expected: t1=2, t2=3, t3=4  
; =====  
; =====  
; TEST 2: WB→EX forwarding (distance 2)  
; t0 produced in EX, 1 instruction gap, consumed when t0 is in WB  
; without forwarding: t0 not yet committed to regfile → stale read  
; expected: t1=11  
; =====  
; =====  
; TEST 3: back-to-back MEM→EX dependency chain on both rd and rs  
; ADD t0,t0 reads t0 as both operands – both need forwarding simultaneously  
; expected: t0 = 1→2→4→8→16  
; =====  
; =====  
; TEST 4: two independent forwards into the same instruction  
; t2 and t3 explicitly zeroed to avoid residue from previous tests  
; ADD t2,t0: t2=0+3=3 (t0 WB→EX); ADD t3,t1: t3=0+5=5 (t1 MEM→EX)  
; ADD t2,t3: t2=3+5=8 (t2 MEM→EX, t3 WB→EX)  
; expected: t2=8, t3=5  
; =====  
; =====  
; TEST 5: forwarding with SUB and CMP  
; t2=8 explicitly loaded to isolate this test from TEST 4 residue  
; SUB t2,t0: t2 = t2-t0 = 8-3 = 5 (t0 forwarded WB→EX)  
; CMP t2,t1: CMP(5, 5) → Z=1 (t2 forwarded MEM→EX, t1 forwarded WB→EX)  
; expected: Z=1 N=0  
; =====
```

Test Code forwarding_00.asm

```

reset:
    LI    sp, #STACK_TOP          ; => IMM #0x03F      [8|03F] = 0x803F (prefix: upper 12 bits of STACK_TOP=0x03FF)      | MEM_ADDR = 0x0100
                                           ; => ADDI sp,zero,#0xF    [1|D|0|F] = 0x1D0F (sp = 0x03FF with prefix)      | MEM_ADDR = 0x0102

main:
    ; Deactivate TimerH int_en
    IMM   #0x408                  ; [8|408] = 0x8408 (prefix: upper 12 bits of MMIO word-address 0x8100)      | MEM_ADDR = 0x0104
    SW    r0, r0, #0              ; [6|0|0|0] = 0x6000 (MEM[0+0x8100] = r0 = 0, using prefix)      | MEM_ADDR = 0x0106

    ; =====
    ; TEST 1: MEM→EX forwarding (distance 1)
    ; t0 produced in EX, consumed next cycle when t0 is in MEM
    ; without forwarding: t0 still in EX → regfile read returns stale value
    ; expected: t1=2, t2=3, t3=4
    ; =====
    LI    t0, #1                  ; => IMM #0x000      [8|000] = 0x8000 (prefix: 1>>4 = 0)      | MEM_ADDR = 0x0108
                                           ; => ADDI t0,zero,#0x1  [1|4|0|1] = 0x1401 (t0 = 1)      | MEM_ADDR = 0x010A

    ADDI  t1, t0, #1              ; [1|5|4|1] = 0x1541 (t1 = t0+1 = 2 - t0 forwarded MEM→EX)      | MEM_ADDR = 0x010C
    ADDI  t2, t1, #1              ; [1|6|5|1] = 0x1651 (t2 = t1+1 = 3 - t1 forwarded MEM→EX)      | MEM_ADDR = 0x010E
    ADDI  t3, t2, #1              ; [1|7|6|1] = 0x1761 (t3 = t2+1 = 4 - t2 forwarded MEM→EX)      | MEM_ADDR = 0x0110

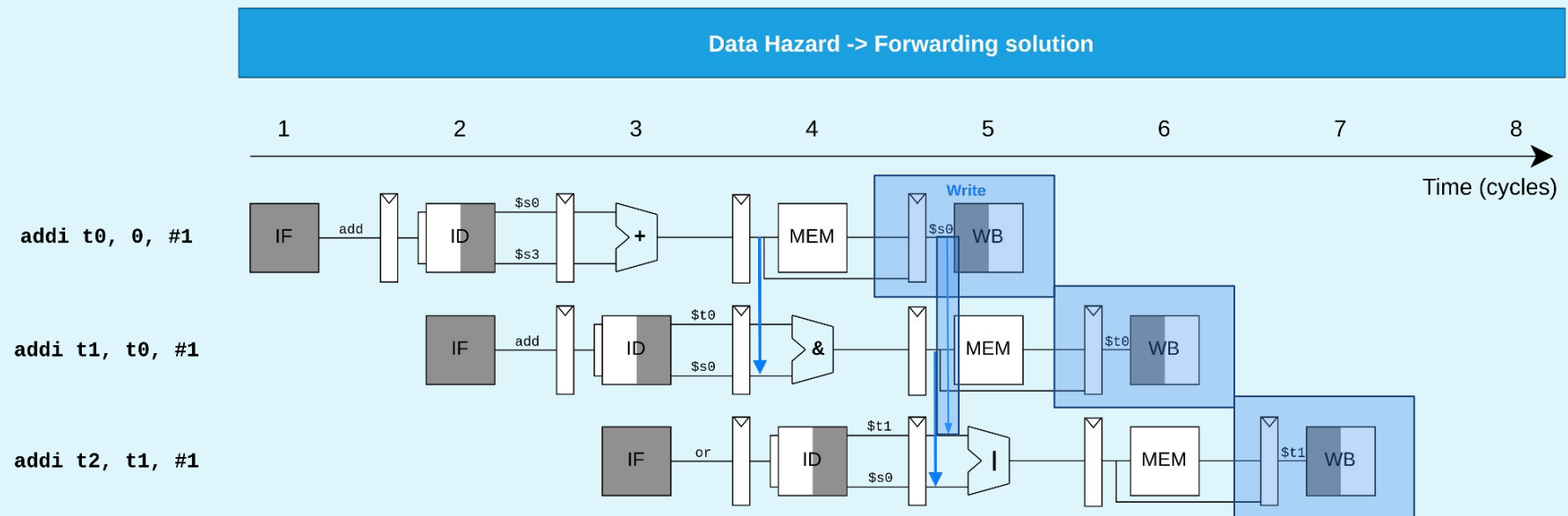
    ; =====
    ; TEST 2: WB→EX forwarding (distance 2)
    ; t0 produced in EX, 1 instruction gap, consumed when t0 is in WB
    ; without forwarding: t0 not yet committed to regfile → stale read
    ; expected: t1=11
    ; =====
    LI    t0, #10                 ; => IMM #0x000      [8|000] = 0x8000 (prefix: 10>>4 = 0)      | MEM_ADDR = 0x0112
                                           ; => ADDI t0,zero,#0xA  [1|4|0|A] = 0x140A (t0 = 10)      | MEM_ADDR = 0x0114

    ADDI  s0, s0, #0              ; [1|8|8|0] = 0x1880 (gap instruction, does not use t0)      | MEM_ADDR = 0x0116
    ADDI  t1, t0, #1              ; [1|5|4|1] = 0x1541 (t1 = t0+1 = 11 - t0 forwarded WB→EX)      | MEM_ADDR = 0x0118

```

(...) (continues)

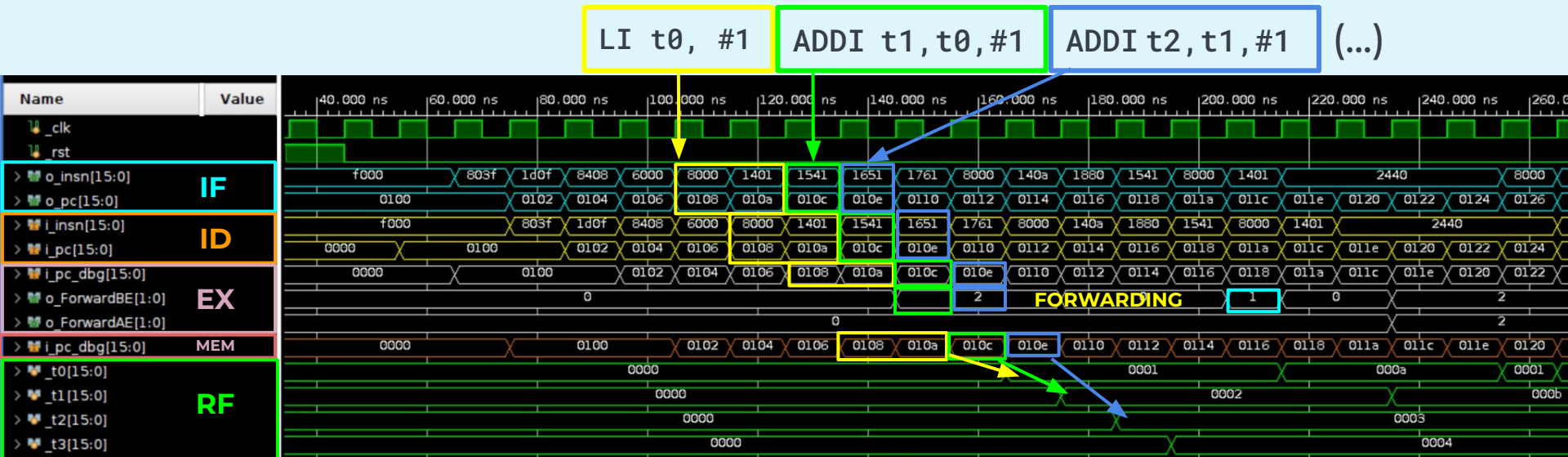
Diagram of the Scenario



Instructions
from Test 1

In the validation of the implementation we are looking for the **forwarding register status**, which in the **diagram** is represented by the **blue arrows**.

Forwarding Validation

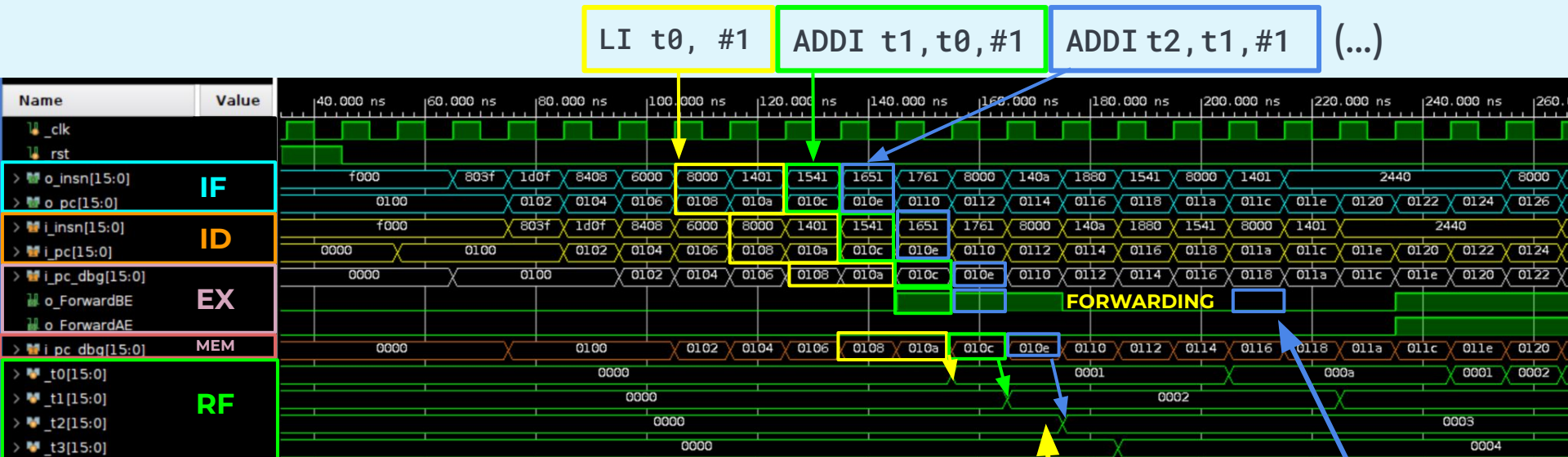


Observations

1. Values are written on the RF on WB stage, so, the cycle after WB, the new value appears in the RF.
2. The forwarding demonstrated in the image is a MEM→EX forwarding, where the value required in instruction N is being used by instruction N-1
3. The rest of the test code validates other circumstances

Forwarding

Validation — Comparison with No WB



Values are written earlier in the RF

There is no longer need to forward from WB stage

03.3

Control Hazards

The pipeline implements:

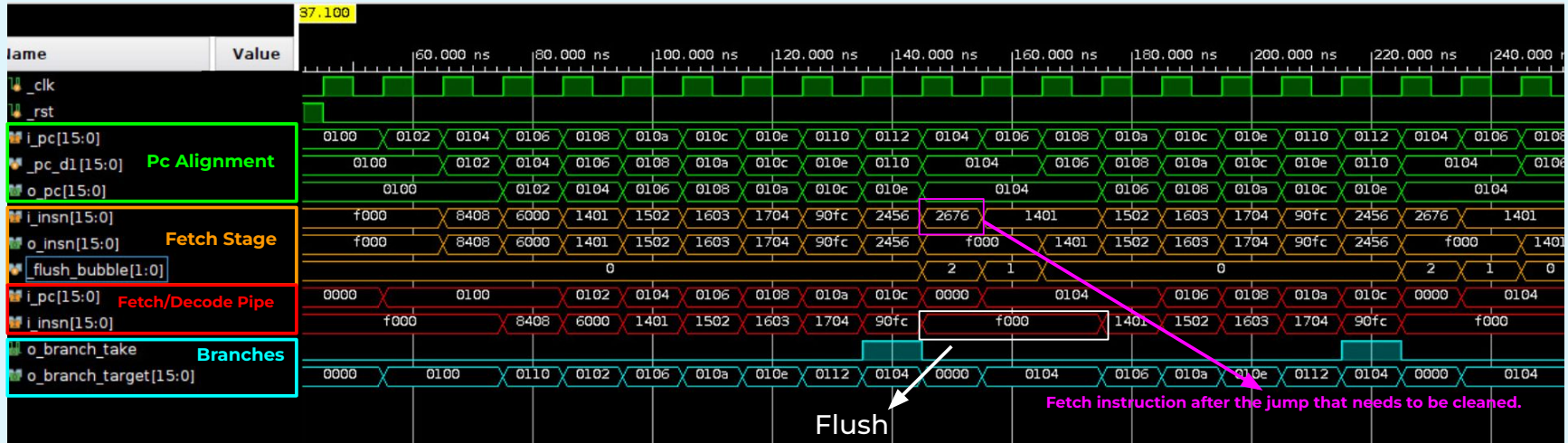
- ❖ **Early-branch decision**
- ❖ ***predict-not-taken***

Branches are **evaluated** early in the pipeline stage, in the **ID stage**.

When a branch is taken, the **branch misprediction penalty** is 2 (clock cycles).

Pipeline Behavior with Not Taken Branches

Branch Always → o_branch_take is activated and flush occurs.



```

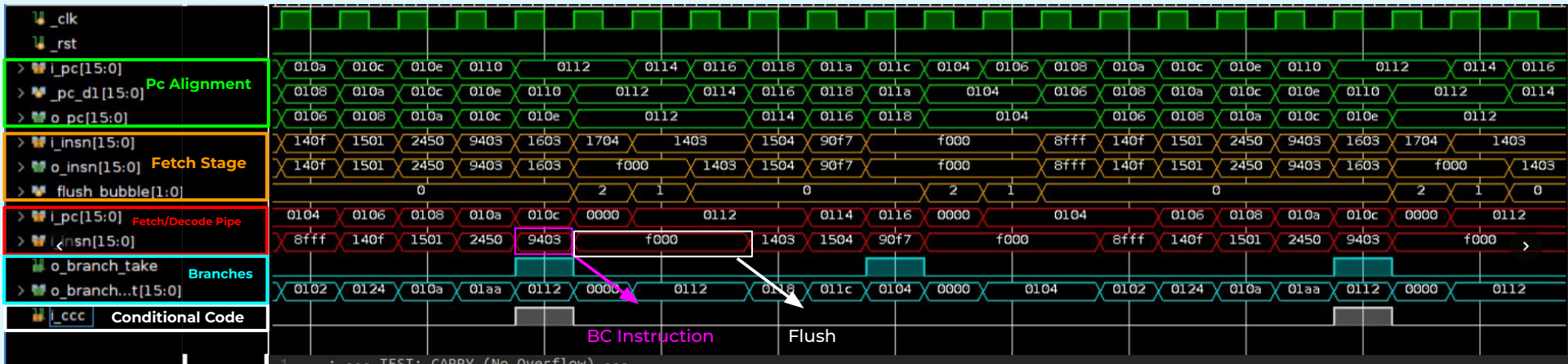
1 .equ TIMER1_HI, 0x408
2
3 .org 0x0100
4
5 reset:
6
7 ; Turns off timer1 (int_en=0, timer_mode=0)
8
9
10
11
12 IMM #TIMER1_HI ; [8|408] = 0x8408 (prefix: upper 12 bits of MMIO word-address 0x8100) |MEM_ADDR = 0x0100
13 SW r0, r0, #0 ; [6|0|0|0] = 0x6000 (MEM[0+0x8100] = r0 = 0, using prefix) |MEM_ADDR = 0x0102
14
15
16
17 main:
18 ADDI t0, zero, #1 ; t0 = 1 ; [1|4|0|1] = 0x1401 |MEM_ADDR = 0x0104
19 ADDI t1, zero, #2 ; t1 = 2 ; [1|5|0|2] = 0x1502 |MEM_ADDR = 0x0106
20 ADDI t2, zero, #3 ; t2 = 3 ; [1|6|0|3] = 0x1603 |MEM_ADDR = 0x0108
21 ADDI t3, zero, #4 ; t3 = 4 ; [1|7|0|4] = 0x1704 |MEM_ADDR = 0x010A
22
23 BR main ; [9|0|FC] = 0x90FC (disp=-4) |MEM_ADDR = 0x010C
24
25 ;random instructions
26
27 CMP t0,t1 ; [2|4|5|6] = 0x2456 |MEM_ADDR = 0x010E
28 CMP t2,t3 ; [2|6|7|6] = 0x2676 |MEM_ADDR = 0x0110
29 ADDI t0, zero, #1 ; t0 = 1 ; [1|4|0|1] = 0x1401 |MEM_ADDR = 0x0112
30

```

Pipeline Behavior with Not Taken Branches



Branch If Carry → o_branch_take is activated when an operation generates a Carry, and after flush occurs.



jumping condition:

BR_BC: _t = i_ccc;

```

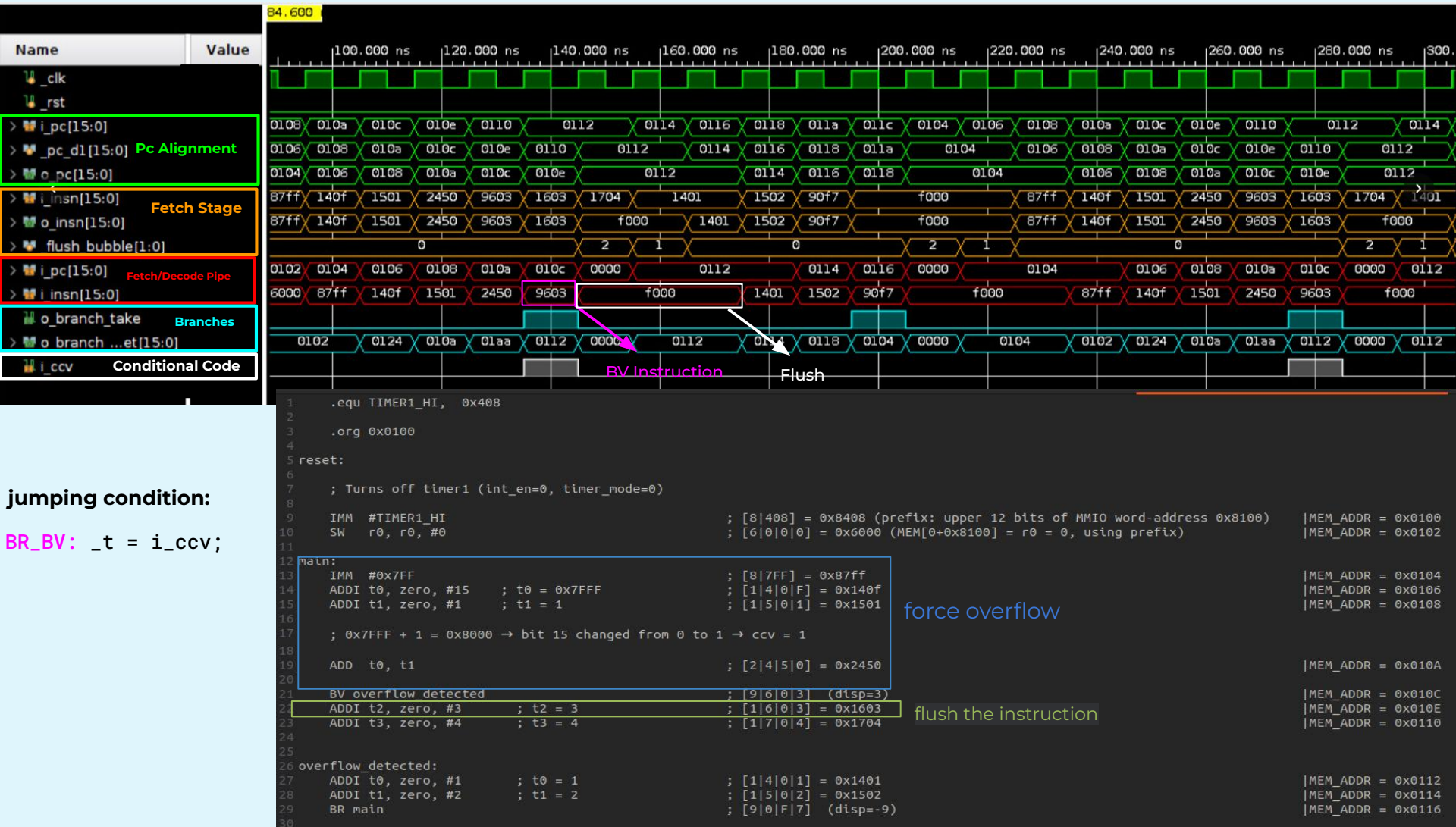
1  ; --- TEST: CARRY (No Overflow) ---
2
3  .equ TIMER1_HI, 0x408
4
5  .org 0x0100
6
7 reset:
8     ; Turns off timer1 (int_en=0, timer_mode=0)
9
10    IMM #TIMER1_HI          ; [8|408] = 0x8408 (prefix: upper 12 bits of MMIO word-address 0x8100) |MEM_ADDR = 0x0100
11    SW r0, r0, #0          ; [6|0|0|0] = 0x6000 (MEM[0+0x8100] = r0 = 0, using prefix) |MEM_ADDR = 0x0102
12
13 main:
14    ;0xFFFF (-1 in signed, 65535 in unsigned) + 1    carry uses unsigned values    generate a carry on the sum
15    IMM #0xFFFF          ; [8|FFF] |MEM_ADDR = 0x0104
16    ADDI t0, zero, #15    ; [1|4|0|F] |MEM_ADDR = 0x0106
17    ADDI t1, zero, #1     ; t1 = 1          ; [1|5|0|1] |MEM_ADDR = 0x0108
18    ADD t0, t1            ; 0xFFFF + 1 → ccc=1, ccv=0 ;[2|4|5|0] |MEM_ADDR = 0x010A
19    BC test_bc           ; [9|4|0|3] (disp=3) |MEM_ADDR = 0x010C
20
21    ;random instructions
22    ADDI t2, zero, #3     ; t2 = 3          ; [1|6|0|3] = 0x1603    flush the instruction |MEM_ADDR = 0x010E
23    ADDI t3, zero, #4     ; t3 = 4          ; [1|7|0|4] = 0x1704 |MEM_ADDR = 0x0110
24
25
26 test_bc:
27
28    ADDI t0, zero, #3     ; t2 = 3          ; [1|4|0|3] = 0x1603 |MEM_ADDR = 0x0112
29    ADDI t1, zero, #4     ; t3 = 4          ; [1|5|0|4] = 0x1704 |MEM_ADDR = 0x0114
30 BR main                ; [9|0|F7] (disp=-9) |MEM_ADDR = 0x0116
31
32

```

Pipeline Behavior with Not Taken Branches



Branch If Overflow → `o_branch_take` is activated when an overflow occurs, and after flush occurs.

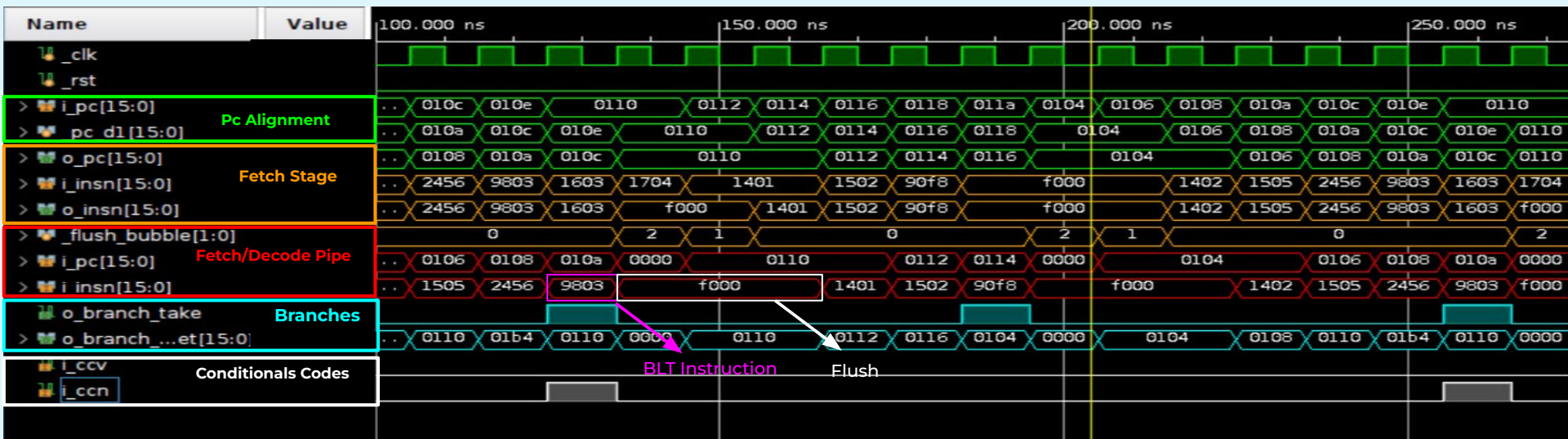


Pipeline Behavior with Not Taken Branches



CENTROALGORITMI

Branch If Less That → `o_branch_take` is activated through the XOR logical operation between the Negative and Overflow Conditions Codes



jumping condition:

BR_BLT: `_t = i_ccn ^ i_ccv;`

```

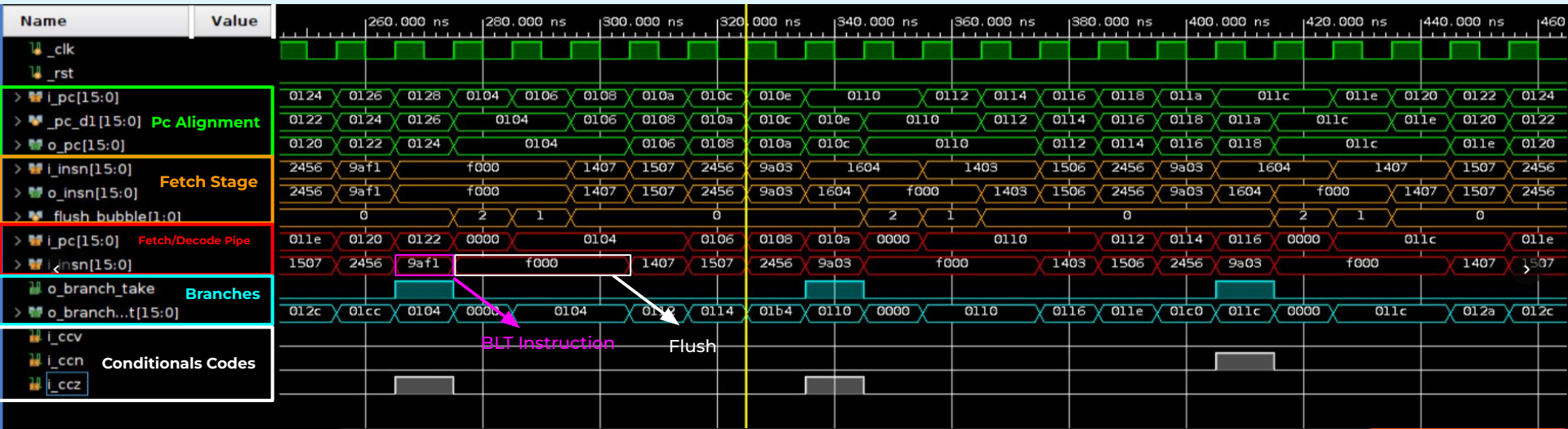
1
2 .equ TIMER1_HI, 0x408
3
4 .org 0x0100
5
6 reset:
7
8 ; Turns off timer1 (int_en=0, timer_mode=0)
9
10 IMM #TIMER1_HI ; [8|408] = 0x8408 (prefix: upper 12 bits of MMIO word-address 0x8100) | MEM_ADDR = 0x0100
11 SW r0, r0, #0 ; [6|0|0|0] = 0x6000 (MEM[0+0x8100] = r0 = 0, using prefix) | MEM_ADDR = 0x0102
12
13 main:
14 ADDI t0, zero, #2 ; t0 = 2 ; [1|4|0|2] | MEM_ADDR = 0x0104
15 ADDI t1, zero, #5 ; t1 = 5 ; [1|5|0|5] | MEM_ADDR = 0x0106
16 CMP t0, t1 ; 2 < 5 ccn = 1 & ccv = 0 ; [2|4|5|0] Comparison | MEM_ADDR = 0x0108
17 BLT test_blt ; [9|8|0|3] (disp=3) | MEM_ADDR = 0x010A
18
19 ADDI t2, zero, #3 ; t2 = 3 ; [1|6|0|3] = 0x1603 flush the instruction | MEM_ADDR = 0x010C
20 ADDI t3, zero, #4 ; t3 = 4 ; [1|7|0|4] = 0x1704 | MEM_ADDR = 0x010E
21
22
23 test_blt:
24 ADDI t0, zero, #1 ; t0 = 1 ; [1|4|0|1] = 0x1401 | MEM_ADDR = 0x0110
25 ADDI t1, zero, #2 ; t1 = 2 ; [1|5|0|2] = 0x1502 | MEM_ADDR = 0x0112
26 BR main ; [9|0|F|8] (disp=-8) | MEM_ADDR = 0x0114
27
28
29

```

Pipeline Behavior with Not Taken Branches



Branch If Less or Equal → o_branch_take is activated similar to the previous one but it can be the equal (ccz = 1)



jumping condition:

BR_BLE: $t = (i_ccn \wedge i_ccv) \mid i_ccz$

```
.equ TIMER1_HI, 0x408
.org 0x0100
reset:
    ; Turns off timer1 (int_en=0, timer_mode=0)
    IMM #TIMER1_HI
    SW r0, r0, #0
main:
    ADDI t0, zero, #7 ; t0 = 7 ; [1|4|0|7] ; MEM_ADDR = 0x0104
    ADDI t1, zero, #7 ; t1 = 7 ; [1|5|0|7] ; MEM_ADDR = 0x0106
    CMP t0, t1 ; 7 - 7 = 0 ; [2|4|5|6] ; MEM_ADDR = 0x0108
    BLE success_equal ; [9|A|03] (disp=3) ; MEM_ADDR = 0x010A
    ;random instructions
    ADDI t2, zero, #4 ; [1|4|0|3] ; MEM_ADDR = 0x010C
    ADDI t2, zero, #4 ; [1|4|0|3] ; MEM_ADDR = 0x010E
success_equal:
    ADDI t0, zero, #3 ; t0 = 3 ; [1|4|0|7] ; MEM_ADDR = 0x0110
    ADDI t1, zero, #6 ; t1 = 6 ; [1|5|0|7] ; MEM_ADDR = 0x0112
    CMP t0, t1 ; 3 - 6 = -3 ; [2|4|5|6] ; MEM_ADDR = 0x0114
    BLE success_less ; [9|A|03] (disp=3) ; MEM_ADDR = 0x0116
    ;random instructions
    ADDI t2, zero, #4 ; [1|4|0|3] ; MEM_ADDR = 0x0118
    ADDI t2, zero, #4 ; [1|4|0|3] ; MEM_ADDR = 0x011A
success_less:
    ADDI t0, zero, #7 ; t0 = 7 ; [1|4|0|7] ; MEM_ADDR = 0x011C
    ADDI t1, zero, #7 ; t1 = 7 ; [1|5|0|7] ; MEM_ADDR = 0x011E
    CMP t0, t1 ; 3 - 6 = -3 ; [2|4|5|6] ; MEM_ADDR = 0x0120
    BLE main ; [9|A|F|1] (disp=-15) ; MEM_ADDR = 0x0122
```

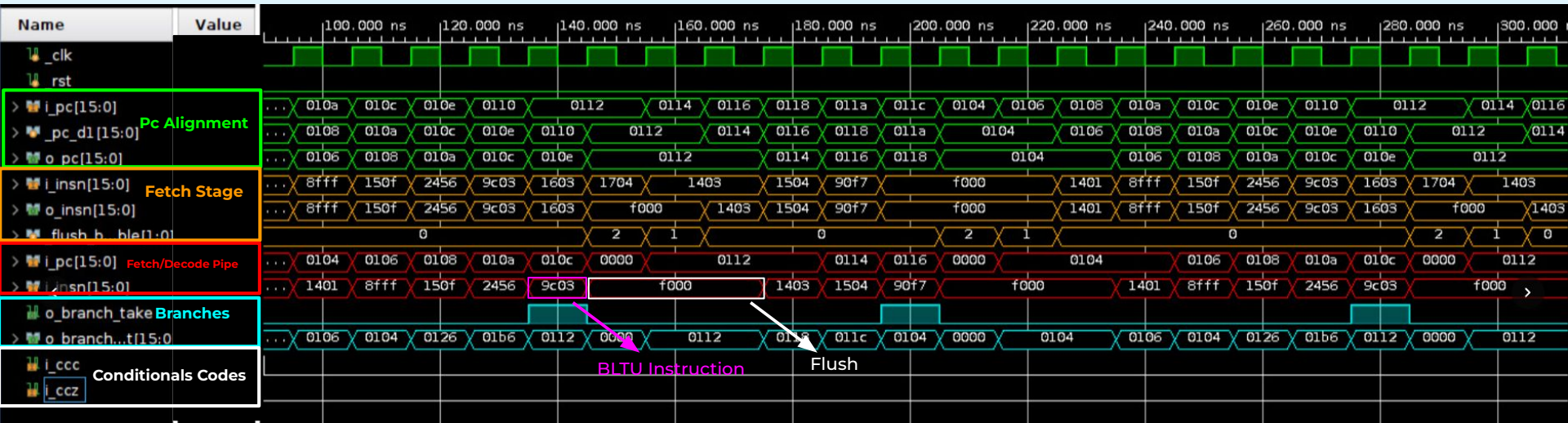
flush the instruction

Equal test

Less Test

Pipeline Behavior with Not Taken Branches

Branch if Less Than Unsigned → o_branch_take is activated for values less than no sign



jumping condition:

BR_BLTU: `_t = ~i_ccz & ~i_ccc;`

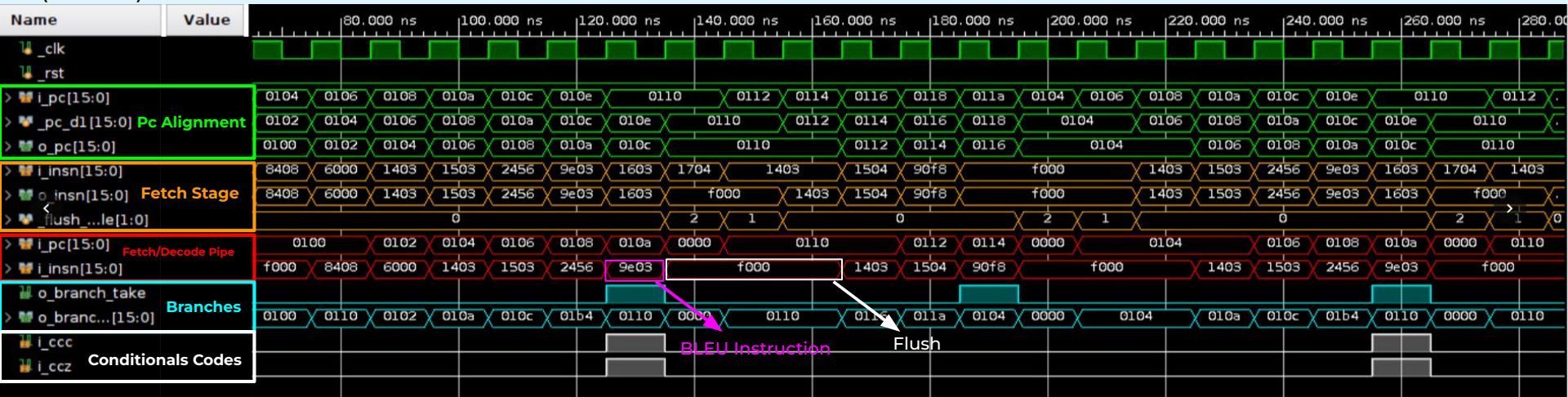
```
1 .equ TIMER1_HI, 0x408
2 .org 0x0100
3
4 reset:
5
6 ; Turns off timer1 (int_en=0, timer_mode=0)
7
8 IMM #TIMER1_HI ; [8|408] = 0x8408 (prefix: upper 12 bits of MMIO word-address 0x8100) | MEM_ADDR = 0x0100
9 SW r0, r0, #0 ; [6|0|0] = 0x6000 (MEM[0+0x8100] = r0 = 0, using prefix) | MEM_ADDR = 0x0102
10
11 main:
12 ADDI t0, zero, #1 ; t0 = 1
13 IMM #0xFFFF
14 ADDI t1, zero, #15 ; t1 = 0xFFFF
15 CMP t0, t1 ; 1 - 65535 → ccc=0 (borrow)
16 BLTU test_bltu
17
18 ;random instructions
19 ADDI t2, zero, #3 ; t2 = 3 ; [1|6|0|3] = 0x1603 | MEM_ADDR = 0x010E
20 ADDI t3, zero, #4 ; t3 = 4 ; [1|7|0|4] = 0x1704 | MEM_ADDR = 0x0110
21
22 test_bltu:
23
24 ;random instructions
25 ADDI t0, zero, #3 ; t0 = 3 ; [1|6|0|3] = 0x1603 | MEM_ADDR = 0x010E
26 ADDI t1, zero, #4 ; t1 = 4 ; [1|7|0|4] = 0x1704 | MEM_ADDR = 0x0110
27 BR main ; [9|0|F7] (disp=-9) | MEM_ADDR = 0x0116
28
29
```

Forces Borrow

flush the instruction

Pipeline Behavior with Not Taken Branches

Branch if Less or Equal Unsigned → o_branch_take is activated similar to the previous one but it can be the e (ccz = 1)



jumping condition:

BR_BLEU: _t = i_ccz | ~i_ccc;

```
1 .include "../tools/abi.inc"
2
3 .equ TIMER1_HI, 0x408
4 .org 0x0100
5
6 reset:
7
8 ; Turns off timer1 (int_en=0, timer_mode=0)
9
10 IMM #TIMER1_HI ; [8|408] = 0x8408 (prefix: upper 12 bits of MMIO word-address 0x8100) | MEM_ADDR = 0x0100
11 SW r0, r0, #0 ; [6|0|0] = 0x6000 (MEM[0+0x8100] = r0 = 0, using prefix) | MEM_ADDR = 0x0102
12
13 ; Conditional code tests
14 main:
15 ; --- Test 1: BEQ (Branch if Equal) ---
16
17 ADDI t0, zero, #5 ; t0 = 5 ; [1|4|0|1] = 0x1401 | MEM_ADDR = 0x0104
18 ADDI t1, zero, #5 ; t1 = 5 ; [1|5|0|2] = 0x1502 | MEM_ADDR = 0x0106
19 CMP t0, t1 ; t0 - t1 = 0 -> CCZ=1 ; [2|4|5|6] = 0x2456 | MEM_ADDR = 0x0108
20
21 BEQ BEQ_test ; [9|2|0|3] = 0x9203 (disp=3) | MEM_ADDR = 0x010A
22
23 ;random instructions
24
25 ADDI t2, zero, #5 ; t2 = 5 ; [1|6|0|3] = 0x1603 | MEM_ADDR = 0x010C
26 ADDI t3, zero, #5 ; t3 = 5 ; [1|7|0|4] = 0x1704 | MEM_ADDR = 0x010E
27
28
29 BEQ_test:
30
31 ADDI t0, zero, #15 ; t0 = 0xF ; [1|4|0|F] = 0x140F | MEM_ADDR = 0x0110
32 ADDI t1, zero, #1 ; t1 = 0x1 ; [1|5|0|1] = 0x1501 | MEM_ADDR = 0x0112
33 BR main ; [9|0|F|8] = 0x90F8 (disp=-8) | MEM_ADDR = 0x0114
34
```

04

CPI

Cycles Per Instruction (CPI)

Average number of clock cycles required for a microprocessor to execute and retire a single instruction.

In a pipelined processor, CPI is considered after $N+1$ clock cycles, where N is the stage number.

$$\text{CPI} = \frac{\text{Total Clock Cycles}}{\text{Number of Instructions}}$$

Non-pipelined

Instruction	Clock cycles
Load (LW/LB)	2
All the others (arithmetic, logic, store, branch, jump)	1

Pipelined

Instruction	Clock cycles
Load (LW/LB) – Memory Resource Stall	2
All the others (arithmetic, logic, store, branch, jump) if there is no hazard	1

Hazard Type	Delay (Cycles)
Data Hazard (RAW - ALU)	0 cycles
Data Hazard (Load-Use)	+ 1 cycle
Control Hazard (Branch Taken / Jump)	+ 2 cycles
Control Hazard (Branch Not Taken)	0 cycles

$$1 \leq \text{CPI}_{\text{NON-PIPELINED}} \leq 2,$$

where $(\text{CPI}_{\text{NON-PIPELINED}} - 1) \times \text{IN} = \text{load_IN},$

considering IN the number of instructions in the program and load_IN the number of load instructions in the program.

$$\text{CPI}_{\text{PIPELINED}} \geq \text{CPI}_{\text{NON-PIPELINED}}$$

We will calculate different instruction combinations and consider 3 scenarios in order to compare the CPIs of them.

CPI Evaluation

Non-Pipelined Processor vs Pipelined Processor

CPI Calculation - Scenario 1 - No hazards

	Load	Store	Branch	ALU
Scenario 1 (No Hazards)	25%	10%	10%	55%

Branch: 100% not taken

Non-Pipelined Processor

$$\text{CPI} = (0.25 * 2) + (0.1 * 1) + (0.1 * 1) + (0.55 * 1) = 1.25$$

Pipelined Processor

$$\text{CPI} = (0.25 * 2) + (0.1 * 1) + (0.1 * 1) + (0.55 * 1) = 1.25$$

CPI Evaluation

Non-Pipelined Processor vs Pipelined Processor

Non-Pipelined Processor

$$\text{CPI} = (0.25 * 2) + (0.1 * 1) + (0.1 * 1) + (0.55 * 1) = 1.25$$

CPI Calculation - Scenario 2 - mixed

	Load	Store	Branch	ALU
Scenario 2 (mix)	25%	10%	10%	55%

Load : 45% immediately followed by an instruction that uses the result

Branch: 25% are taken

ALU : 50% RAW

Pipelined Processor

$$\text{(Load) } \rightarrow \text{CPI} = (0.45 * 3) + (0.55 * 2) = 2.45$$

$$\text{(Branch) } \rightarrow \text{CPI} = (0.25 * 3) + (0.75 * 1) = 1.5$$

$$\text{CPI} = (0.25 * 2.45) + (0.1 * 1) + (0.1 * 1.5) + (0.55 * 1) = 1.41$$

CPI Evaluation

Non-Pipelined Processor vs Pipelined Processor

Non-Pipelined Processor

$$\text{CPI} = (0.25 * 2) + (0.1 * 1) + (0.1 * 1) + (0.55 * 1) = 1.25$$

CPI Calculation - Scenario 3 - Full Hazards

	Load	Store	Branch	ALU
Scenario 3 (full hazards)	25%	10%	10%	55%

Load : 100% immediately followed by an instruction that uses the result

Branch: 100% taken

ALU : 100 % with dependency between instruction (RAW)

Pipelined Processor

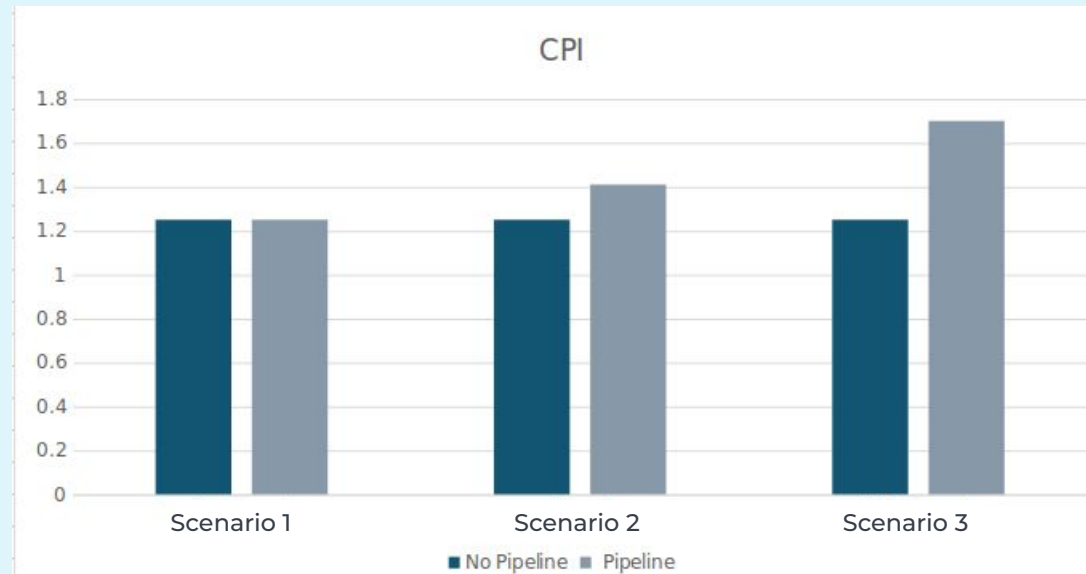
$$\text{CPI} = (0.25 * 3) + (0.1 * 1) + (0.1 * 3) + (0.55 * 1) = 1.7$$

CPI Evaluation

Non-Pipelined Processor vs Pipelined Processor

Final Conclusion

	No Pipeline	Pipeline
Scenario 1	1.25	1.25
Scenario 2	1.25	1.41
Scenario 3	1.25	1.7



The more hazards occur, the higher the CPI on the pipelined processor. This represents a trade-off when pipelining a processor. However, **pipelining allows a higher clock frequency**, since each stage has a shorter critical path, ultimately leading to **higher throughput**.

05

Conclusion

Conclusion

- ❖ Implemented all the pipeline stages
 - Connected them, dealing with the hazards (RAW, load-use and taken-branch) with stalls and flushes
- ❖ Implemented the hazard unit, with forwarding control and load-use stall
- ❖ Implemented *early branch* prediction + *predict-not-taken* logic
- ❖ Validated, in the behavioral simulation, all the hazards and the processor's correct behavior
- ❖ Calculated the CPI of the pipelined processor vs the previous non-pipelined processor

Next Steps

- ❑ Update the designs to not have the WB stage
- ❑ Do the instruction analysis required by the professor according to the book
- ❑ Evaluate Post-Synthesis Timing Simulation and Post-Implementation Simulation
- ❑ Increase CLK cycle (main objective of the pipeline implementation) and evaluate WNS/WHIS – evaluate latency
- ❑ Compare Pipelined processor throughput with the non-pipelined processor throughput
- ❑ Update ILA with important signals

- ❑ Critical path in condition code update from EX to CPU, to be used in ID stage for Bx instructions
 - ❑ This will require changing how the condition codes are updated: a solution is provided in branch pipeline/5-stage-with-stalls, where condition codes are only updated at the end of that instructions pipeline cycle, and travel through the pipeline.
- ❑ Critical path when receiving a value from BRAM and writing it to the RF in the MEM stage (a 5-stage pipeline is also implemented as a fallback).
 - ❑ This analysis considers the case where the destination register (rd) of a load is used as either a source (rs) or destination (rd) in an instruction (non-load) that occurs two cycles after the load (i.e., not the immediately following instruction).

Books

- ❖ (simpler) Read Digital Design and Computer Architecture, H&H, chapter 7 (Microarchitecture) – most important sections are 7.3 (Single Cycle Processor) and 7.5 (Pipelined processor)
http://www.r-5.org/files/books/computers/hw-layers/hardware/digital-design/David_Harris_Sarah_Harris-Digital_Design_and_Computer_Architecture-EN.pdf
Considerations: in the pipeline section the book considers 5-stage pipeline, *write-first* RF and synchronous BRAM with combinational read (!= from what we have, so they only insert one stall cycle in load-use hazard, we need two)
- ❖ Computer Architecture – A Quantitative Approach 4th Edition, by John L. Hennessy and David A. Patterson, Chapters 2 (Instruction-Level Parallelism and Its Exploitation) and 3 (Limits on Instruction-Level Parallelism), Appendix A (Pipelining: Basic and Intermediate Concepts)

- ❖ See the slides
- ❖ See the documents on the Drive (the theoretical are an overview, but you should do something similar so you understand. The others are documentation related to the code and are not updated)
- ❖ Test the various provided test codes and evaluate the execution

Thank you!

Questions?